УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У НОВОМ САДУ

Бане Герић

ЈСД за опис пословних процеса

ЗАВРШНИ РАД

Мастер академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА** 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Бане Герић	Број индекса:	R232/2023
Степен и врста студија:	Мастер академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		

НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:

- проблем – тема рада;
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;

НАСЛОВ ЗАВРШНОГ РАДА:

JСД за опис пословних процеса

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, **РБР**:		
Идентификациони број, **ИБР**:		
Тип документације, **ТД**:		Монографска документација
Тип записа, **ТЗ**:		Текстуални штампани материјал
Врста рада, **ВР**:		Дипломски - мастер рад
Аутор, **АУ**:		Бане Герић
Ментор, **МН**:		Др Игор Дејановић, редовни професор
Наслов рада, **НР**:		ЈСД за опис пословних процеса
Језик публикације, **ЈП**:		српски/ћирилица
Језик извода, **ЈИ**:		српски/енглески
Земља публиковања, **ЗП**:		Република Србија
Уже географско подручје, **УГП**:		Војводина
Година, **ГО**:		2026
Издавач, **ИЗ**:		Ауторски репринт
Место и адреса, **МА**:		Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, **ФО**: (поглавља/страна/ цитата/табела/слика/графика/прилога)		6/75/12/2/28/0/2
Научна област, **НО**:		Електротехничко и рачунарско инжењерство
Научна дисциплина, **НД**:		Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, **ПО**:		Шаблон, завршни рад, упутство
УДК		
Чува се, **ЧУ**:		У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, **ВН**:		
Извод, **ИЗ**:		Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, **ДП**:		
Датум одбране, **ДО**:		01.01.2025
Чланови комисије, **КО**:	Председник:	Др Петар Петровић, ванредни професор
	Члан:	Др Марко Марковић, доцент
	Члан:	Потпис ментора
	Члан, ментор:	Др Игор Дејановић, редовни професор

Образац **Q2.НА.04-05** - Издање 1

UNIVERSITY OF NOVI SAD ● **FACULTY OF TECHNICAL SCIENCES**
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, **ANO**:	
Identification number, **INO**:	
Document type, **DT**:	Monographic publication
Type of record, **TR**:	Textual printed material
Contents code, **CC**:	
Author, **AU**:	Bane Gerić
Mentor, **MN**:	Igor Dejanović, Phd., full professor
Title, **TI**:	DSL for business processes
Language of text, **LT**:	Serbian
Language of abstract, **LA**:	Serbian/English
Country of publication, **CP**:	Republic of Serbia
Locality of publication, **LP**:	Vojvodina
Publication year, **PY**:	2026
Publisher, **PB**:	Author's reprint
Publication place, **PP**:	Novi Sad, Dositeja Obradovica sq. 6
Physical description, **PD**: (chapters/pages/ref./tables/pictures/graphs/appendixes)	6/75/12/2/28/0/2
Scientific field, **SF**:	Electrical and Computer Engineering
Scientific discipline, **SD**:	Applied computer science and informatics
Subject/Key words, **S/KW**:	Template, thesis, tutorial
UC	
Holding data, **HD**:	The Library of Faculty of Technical Sciences, Novi Sad
Note, **N**:	
Abstract, **AB**:	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, **ASB**:	
Defended on, **DE**:	01.01.2025
Defended Board, **DB**: President:	Petar Petrović, Phd., assoc. professor
Member:	Marko Marković, Phd., asist. professor
Member:	
Member, Mentor:	Igor Dejanović, Phd., full professor

Menthor's sign

Obrazac **Q2.HA.04-05** - Izdanje 1

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

_____

Кандидат

_____

Садржај

1 Увод . . . 1
1.1 Мотивација и проблематика . . . 1
1.2 Циљ рада . . . 2
1.3 Методологија . . . 2
1.4 Структура рада . . . 3
2 Теоријски оквир и преглед постојећих рјешења . . . 5
2.1 Језици специфични за домен . . . 5
2.2 Мета-моделовање и моделом вођен развој . . . 6
2.3 Управљање пословним процесима . . . 6
2.4 Формални модели workflow система . . . 7
2.5 Генеративни наспрам интерпретативног приступа . . . 8
2.6 Генерација кода у контексту моделом вођеног развоја . . . 9
2.7 Позиционирање BPML приступа . . . 10
3 Дизајн и архитектура BPML језика . . . 11
3.1 Архитектура система . . . 11
3.2 Формална граматика језика . . . 13
3.3 Структура BPML модела . . . 18
3.4 Семантичка валидација . . . 21
3.5 Дизајнерске одлуке . . . 22
3.6 Алатска подршка: VSCode екстензија . . . 23
3.7 Метамодел и односи елемената . . . 25
4 Генерисање кода . . . 29
4.1 Механизам генерације кода . . . 29
4.2 Генерисање backend система . . . 33
4.3 Генерисање frontend система . . . 37
4.4 Интеграција генерисаних система . . . 41
4.5 Предности аутоматске генерације кода . . . 42
5 Валидација и анализа система . . . 43
5.1 Студија случаја: систем одобравања фактура . . . 43
5.2 Демонстрација генерисаног система . . . 46
5.3 Верификација семантичких провјера . . . 51
5.4 Анализа предности приступа . . . 53
5.5 Поређење са постојећим приступима . . . 54
5.6 Ограничења и потенцијална побољшања . . . 56
5.7 Општа оцјена система . . . 56

6 Закључак 59
6.1 Будући рад 60
Списак слика 63
Списак листинга 65
Списак табела 67
Списак коришћених скраћеница 69
Списак коришћених појмова 71
Биографија 73
Литература 75

Глава 1

Увод

Савремени информациони системи све чешће укључују комплексне пословне процесе који обухватају више актера, стања, правила одлучивања и међусобно зависних активности. Имплементација таквих система класичним програмским приступом доводи до расипања логике процеса кроз више слојева апликације, што отежава одржавање, еволуцију система и формалну анализу понашања.

Насупрот томе, језици специфични за домен (ЈСД, енг. *Domain-Specific Languages* — DSL) омогућавају моделовање проблема на вишем нивоу апстракције, употребом концепата блиских самом домену примјене. Према Дејановићу, језици специфични за домен представљају рачунарске језике који повећавају експресивност ограничавајући се на уско дефинисану област проблема и њене концепте [1]. Овај приступ омогућава већу читљивост, бољу контролу исправности и могућност аутоматске обраде модела.

У области управљања пословним процесима (енг. *Business Process Management* — BPM) често се користе стандардизоване нотације као што је BPMN, али оне углавном захтијевају додатну ручну имплементацију или интеграцију са постојећим системима. Истраживања показују да алати засновани на моделима и генерацији кода могу значајно убрзати развој и смањити број грешака [2]. Управо тај приступ представља основу овог рада.

Централна идеја рада јесте пројектовање и имплементација екстерног језика специфичног за домен, названог BPML (Business Process Modeling Language), који омогућава декларативно дефинисање пословних процеса и аутоматско генерисање комплетне full-stack апликације. Језик омогућава дефинисање ентитета, улога, стања, задатака и транзиција, уз формалну семантичку валидацију модела. На основу једног модела генерише се:

- backend систем заснован на Spring Boot платформи,
- frontend апликација реализована употребом React и TypeScript технологија,
- процесна машина за управљање извршавањем процеса,
- механизам контроле приступа заснован на улогама.

1.1 Мотивација и проблематика

У класичном приступу развоју пословних апликација логика процеса често је имплицитно уграђена у сервисни слој, контролере или чак кориснички интерфејс. Овакав приступ отежава формалну провјеру исправности, увођење измјена и поновну упо-

требу логике у другим системима. Додатно, не постоји јединствена спецификација која би недвосмислено описивала ток процеса.

Постојећи BPM системи попут Camunda или Activiti нуде зрела рјешења за извршавање процеса, али се заснивају на интерпретативном приступу који захтијева засебну runtime инфраструктуру, конфигурацију engine механизма и усклађивање процесног модела са постојећим апликационим кодом. Алтернативни генеративни приступ, гдје се из формалне спецификације аутоматски генерише комплетан систем, остаје недовољно истражен у контексту пословних процеса.

Мотивација за развој BPML језика произилази из потребе за:

- јасном и формалном спецификацијом пословних процеса,
- централизацијом процесне логике у једном моделу,
- аутоматском провјером семантичке исправности прије генерације кода,
- смањењем количине ручно писаног инфраструктурног кода,
- убрзањем развоја прототипа и продукционих система.

1.2 Циљ рада

Основни циљ рада је пројектовање, имплементација и анализа језика специфичног за домен намијењеног моделовању пословних процеса, као и развој генератора кода који из датог модела производи функционалну full-stack апликацију.

Специфични циљеви рада су:

- дефинисање формалне граматике језика употребом TextX алата,
- имплементација семантичких провјера модела,
- дизајн архитектуре генератора кода,
- генерисање backend система заснованог на Spring Boot технологији,
- генерисање frontend система заснованог на React и TypeScript технологијама,
- имплементација процесне машине за управљање стањима и задацима,
- развој алатске подршке за језик у виду VSCode екстензије,
- анализа предности и ограничења предложеног приступа.

1.3 Методологија

У раду је примијењен приступ заснован на моделом вођеном развоју (енг. *Model-Driven Development* — MDD). Граматика језика дефинисана је употребом TextX метајезика, који омогућава генерисање метамодела и апстрактног синтаксног стабла на основу текстуалне спецификације [2].

Након синтаксне анализе, над моделом се извршавају семантичке провјере које обезбјеђују:

- јединственост назива елемената,

- исправност референци између ентитета,
- валидност транзиција између стања,
- исправну употребу улога и задатака.

Генерисање кода реализовано је употребом Jinja2 шаблонског механизма, интегрисаног са TextX екосистемом путем библиотеке `textx-jinja`. На тај начин се једном дефинисана спецификација процеса преводи у потпуно функционалан софтверски систем. Валидација предложеног приступа спроведена је кроз студију случаја, моделовање и генерисање система за одобравање фактура, којом је демонстрирана функционална исправност генерисаног система.

1.4 Структура рада

Рад је организован у шест поглавља.

У другом поглављу дат је теоријски оквир који обухвата концепте језика специфичних за домен, моделом вођеног развоја и управљања пословним процесима. Такође је дато поређење генеративног и интерпретативног приступа имплементацији процеса, укључујући преглед постојећих система попут Camunda, Activiti и jBPM.

Треће поглавље описује дизајн и архитектуру BPML језика: формалну граматику, структуру модела, семантичку валидацију и кључне дизајнерске одлуке.

Четврто поглавље анализира механизам генерисања кода, архитектуру генерисаног backend система заснованог на Spring Boot платформи, архитектуру генерисаног frontend система заснованог на React и TypeScript технологијама, те интеграцију оба слоја.

У петом поглављу представљена је валидација система кроз студију случаја (систем одобравања фактура), демонстрација семантичких провјера на примјерима невалидних модела, поређење са постојећим приступима и анализа предности и ограничења.

Шесто поглавље садржи закључак и правце будућег развоја.

Глава 2

Теоријски оквир и преглед постојећих рјешења

Развој сложених информационих система захтијева јасну апстракцију доменских концепата, формализацију пословне логике и могућност аутоматске трансформације спецификације у извршиву имплементацију. У том контексту, језици специфични за домен, моделом вођен развој и системи за управљање пословним процесима представљају кључне теоријске основе овог рада.

2.1 Језици специфични за домен

Језици специфични за домен (ЈСД, енг. *Domain-Specific Languages* — DSL) представљају рачунарске језике чија је експресивност усмјерена на уско дефинисану проблемску област. За разлику од језика опште намјене, који настоје да буду универзални, DSL-ови ограничавају синтаксу и семантику на појмове релевантне одређеном домену.

Фаулер дефинише DSL као језик намијењен изражавању решења унутар специфичног проблемског простора, при чему се повећава читљивост и смањује семантички јаз између домена и имплементације [3]. Мерник и сарадници истичу да DSL-ови омогућавају повећање продуктивности, смањење грешака и бољу комуникацију између доменских експерата и програмера [4].

Према класификацији, DSL-ови могу бити:

- интерни (уграђени у постојећи језик),
- екстерни (са сопственом синтаксом и граматиком).

Интерни DSL-ови реализују се као библиотеке или API-ји унутар постојећег језика опште намјене, наслијеђујући његову синтаксу и алате. Предност овог приступа јесте једноставност имплементације и директна доступност екосистема домаћег језика. Међутим, интерни DSL-ови су ограничени синтаксним правилима језика у који су уграђени, што често онемогућава постизање потпуно природне доменске нотације.

Екстерни DSL-ови, са друге стране, дефинишу сопствену граматику и захтијевају имплементацију парсера, семантичке анализе и евентуално генератора кода. Овај приступ омогућава потпуну контролу над синтаксом и независност од ограничења језика опште намјене. Екстерни приступ је погодан када је потребно формално дефинисати структуру модела и обезбиједити строгу семантичку валидацију.

У овом раду обрађен је екстерни DSL, пројектован тако да синтакса и семантика буду у потпуности прилагођене домену пословних процеса.

2.2 Мета-моделовање и моделом вођен развој

Моделом вођен развој (енг. *Model-Driven Development* — MDD) представља парадигму у којој модели имају централну улогу у процесу изградње система. Шмит истиче да MDD омогућава подизање нивоа апстракције и аутоматизацију трансформације модела у имплементацију [5].

Основни концепти укључују:

- метамодел — дефиницију структуре модела,
- модел — конкретну инстанцу метамодела,
- трансформацију — процес превођења модела у други модел или изворни код.

OMG иницијатива Model-Driven Architecture (MDA) формализује овај приступ кроз раздвајање платформски независног модела (енг. *Platform-Independent Model* — PIM) и платформски специфичног модела (енг. *Platform-Specific Model* — PSM). У контексту овог рада, BPML модел представља платформски независну спецификацију, док се Spring Boot и React код могу посматрати као платформски специфичне имплементације.

TextX алат омогућава дефинисање граматике и аутоматско генерисање метамодела на основу текстуалне спецификације [2]. За разлику од традиционалних приступа који захтијевају одвојену дефиницију метамодела и граматике, TextX ове два елемента обједињује у јединственој спецификацији. На основу граматике алат аутоматски гради метамодел, парсира улазни текст и конструише апстрактно синтаксно стабло. Додатно, TextX подржава регистрацију семантичких процесора који се извршавају над парсираним моделом, чиме се омогућава имплементација статичких провјера конзистентности и исправности модела.

2.3 Управљање пословним процесима

Пословни процес дефинише се као скуп међусобно повезаних активности које трансформишу улазне ресурсе у излазне вриједности. Думас и сарадници описују процес као структурирану секвенцу активности са јасно дефинисаним стањима и правилима прелаза [6].

Основни елементи процесног модела су: стања, активности, транзиције, актери и пословни објекти. У пракси, различити формализми различито наглашавају ове елементе. Неки стављају фокус на ток активности, други на промјене стања, а трећи на улоге учесника.

BPMN (енг. *Business Process Model and Notation*) представља најшире прихваћену нотацију за моделовање пословних процеса. Овај стандард, одржаван од стране OMG (Object Management Group), дефинише графичку нотацију за спецификацију процеса и подржава широк спектар контролних образаца, укључујући секвенцијалне токове, паралелне гране, условна гранања, петље и догађаје. BPMN омогућава визуелну спецификацију процеса разумљиву и техничким и пословним стручњацима, али се при извршавању углавном ослања на засебне engine механизме.

Веске истиче да workflow engine мора обезбиједити контролу стања, синхронизацију паралелних активности и управљање улогама [7]. Многи постојећи системи заснивају се на интерпретацији процесног модела током извршавања, што уводи додатни runtime слој између спецификације и имплементације.

2.4 Формални модели workflow система

У области управљања пословним процесима значајан допринос дали су формални модели засновани на Петријевим мрежама. Ван дер Алст показује да Петријеве мреже представљају математички формализам погодан за моделовање стања, паралелизма и синхронизације у workflow системима [8].

Петријева мрежа дефинисана је као бипартитни усмјерени граф који се састоји од мјеста (енг. *places*), која представљају услове или стања, прелаза (енг. *transitions*), који представљају догађаје или активности, и токена (енг. *tokens*), који означавају тренутно стање система. Формална семантика Петријевих мрежа омогућава:

- верификацију процеса (провјеру да ли процес може завршити из сваког достижног стања),
- анализу deadlock ситуација (откривање стања из којих процес не може наставити),
- провјеру достижности стања (да ли је одређено стање могуће постићи),
- анализу паралелизма (моделовање конкурентних активности).

У каснијим радовима, Ван дер Алст проширује анализу кроз концепт *process mining*-а, који омогућава анализу извршених процеса на основу логова из информационих система [9]. Овај приступ допуњује формалну верификацију увидом у стварно понашање система и омогућава откривање одступања између пројектованог и реалног тока процеса.

У контексту BPML језика, концепти стања, транзиција и зависности задатака могу се посматрати као поједностављена апстракција формалних workflow модела. Иако BPML не користи директно Петријеве мреже, његова структура, стања као аналогија мјестима, транзиције као аналогија прелазима, и механизам зависности задатака као форма синхронизације, у складу је са основним принципима формалног моделовања процеса.

2.5 Генеративни наспрам интерпретативног приступа

У области имплементације пословних процеса могу се идентификовати два доминантна приступа: интерпретативни и генеративни. Разлика између ових приступа има дубоке посљедице на архитектуру, перформансе, флексибилност и одржавање система.

2.5.1 Интерпретативни приступ

Код интерпретативног приступа, процесни модел се чува у бази података или конфигурационом фајлу и тумачи током извршавања од стране специјализованог engine механизма. Workflow engine чита дефиницију процеса, одређује тренутно стање, евалуира услове и покреће одговарајуће активности. Све то ради у runtime-у, без компилације модела у изворни код.

Овај приступ доминира у комерцијалним и отвореним BPM платформама. Camunda, настала 2008. године као консултантска фирма, развила је платформу засновану на BPMN 2.0 стандарду. Camunda 7, која је првобитно произашла из Activiti пројекта, функционише као Java оквир са уграђеним BPMN engine механизмом који се може интегрисати у постојеће Java апликације или покренути као самостални сервис. Платформа обезбјеђује REST API за удаљени приступ, веб апликације за управљање задацима и мониторинг процеса, те интеграцију са Spring и Java EE екосистемима. Camunda 8 уводи дистрибуирани engine механизам Zeebe, заснован на обради токова догађаја (енг. *event stream processing*), пројектован за хоризонталну скалабилност и отпорност на грешке у окружењу микросервиса.

Activiti представља отворену BPM платформу развијену под покровитељством Alfresco фондације. Као и Camunda, заснива се на BPMN 2.0 стандарду и функционише унутар Java виртуелне машине. Activiti је имао значајан утицај на развој BPM екосистема. Из њега су произашли како Camunda (која се одвојила 2013. године) тако и Flowable, новија варијанта истог пројекта.

jBPM, развијен у оквиру Red Hat екосистема, представља лаки workflow engine за извршавање пословних процеса дефинисаних у BPMN 2.0. За разлику од Camunda платформе, jBPM је тјесно интегрисан са Drools механизмом за управљање пословним правилима, чиме се омогућава комбиновање процесне логике и логике одлучивања.

Основне предности интерпретативног приступа јесу: могућност динамичке измјене процеса без поновне компилације и deploymenta, зрео екосистем алата за моделовање, мониторинг и анализу, те усклађеност са BPMN стандардом који олакшава комуникацију између техничких и пословних стручњака. Међутим, овај приступ уводи додатни runtime слој који повећава сложеност система, отежава дебаговање и потенцијално уводи додатну латенцију. Такође, постојећи BPM системи захтијевају засебну инфраструктуру, конфигурацију и одржавање engine механизма.

2.5.2 Генеративни приступ

Генеративни приступ подразумијева трансформацију процесног модела у изворни код који се компајлира и извршава као дио апликације. У овом моделу не постоји засебни runtime engine него процесна логика постаје интегрални дио генерисане апликације. Овај приступ је у складу са принципима моделом вођеног развоја [5].

Предности генеративног приступа су:

- боље перформансе, јер не постоји интерпретациони слој,
- јаснија структура кода, будући да је генерисани код директно читљив и подложан стандардним алатима за дебаговање,
- лакше тестирање генерисаних компоненти стандардним тест оквирима,
- директна повезаност спецификације и имплементације, чиме се смањује ризик од одступања.

Основни недостатак овог приступа јесте потреба за поновном генерацијом и компилацијом у случају измјене процеса, што онемогућава динамичко мијењање процесне логике без прекида рада система.

2.5.3 Образложење одабраног приступа

BPML систем свјесно се ослања на генеративни приступ, чиме се позиционира као генеративни DSL систем, а не као класични интерпретативни workflow engine. Ова одлука мотивисана је специфичним циљевима рада: стварање система који интегрише процесну логику директно у апликациони код, поједностављује архитектуру елиминацијом засебног engine слоја, смањује зависност од спољашњих runtime компоненти и обезбјеђује тјесну интеграцију са доменским моделом. За разлику од Camunda или Activiti система који захтијевају посебну конфигурацију и runtime инфраструктуру, BPML производи самосталну апликацију која садржи сву потребну логику.

2.6 Генерација кода у контексту моделом вођеног развоја

Аутоматска генерација кода представља кључни механизам моделом вођеног развоја. Кливленд наглашава да генератори омогућавају елиминацију понављајућих шаблонских дијелова и повећање конзистентности система [10].

У контексту MDD-а, генерација кода реализује се трансформацијом модела у изворни код циљне платформе. Овај процес обично укључује три елемента: структуру модела (апстрактно синтаксно стабло), мапирање типова између домена и циљне платформе и контекст трансформације који обухвата метаподатке о пројекту и конфигурацији.

Шаблонски приступ генерацији (енг. *template-based code generation*) представља једну од најраширенијих техника. У овом приступу, шаблони дефинишу структуру генерисаног кода са означеним мјестима за интерполацију вриједности из модела. Алати попут Jinja2 (Python), FreeMarker (Java) и Xtend (Java/Eclipse) омогућавају дефиниса-

ње параметризованих шаблона са контролним структурама (услови, петље, филтери), чиме се исти шаблон може примијенити на различите елементе модела и произвести различите излазне фајлове.

У овом раду генерација се заснива на Jinja2 шаблонском механизму, интегрисаном са TextX екосистемом путем библиотеке `textx-jinja`. Овај приступ омогућава да једна BPML спецификација резултира комплетном backend и frontend апликацијом, при чему су детаљи генерације описани у четвртом поглављу.

2.7 Позиционирање BPML приступа

На основу анализе теоријског оквира и постојећих рјешења може се закључити да различити приступи наглашавају различите аспекте управљања пословним процесима: BPMN системи доминирају у визуелном моделовању, workflow engine системи омогућавају динамичко извршавање, MDD алати омогућавају трансформацију модела, а DSL алати омогућавају формалну спецификацију домена.

BPML интегрише ове концепте у јединствен систем који:

- користи екстерни DSL за декларативну спецификацију процеса,
- примјењује статичку семантичку валидацију над моделом,
- генерише комплетну full-stack имплементацију (backend и frontend),
- укључује процесну машину и визуелизацију процесног графа.

Овакво позиционирање омогућава да се BPML посматра као практична синтеза DSL и MDD концепата у домену управљања пословним процесима. За разлику од интерпретативних платформи, BPML не захтијева засебну runtime инфраструктуру, а за разлику од чисто визуелних алата, обезбјеђује формалну текстуалну спецификацију подложну верзионисању, аутоматској валидацији и програмској обради.

Глава 3

Дизајн и архитектура BPML језика

BPML (Business Process Modeling Language) представља екстерни језик специфичан за домен намијењен моделовању пословних процеса и аутоматској генерацији full-stack софтверских система. Језик је имплементиран употребом TextX алата [2], док је трансформација модела у изворни код реализована путем Jinja2 шаблонског механизма.

Дизајн језика заснива се на четири кључна принципа: јасној декларативној синтакси која одражава концепте домена пословних процеса, формалном метамоделу који дефинише структуру и односе између елемената, статичкој семантичкој валидацији која обезбјеђује конзистентност модела прије генерације кода, и генерацији кода заснованој на моделу која омогућава аутоматску трансформацију спецификације у извршиву имплементацију.

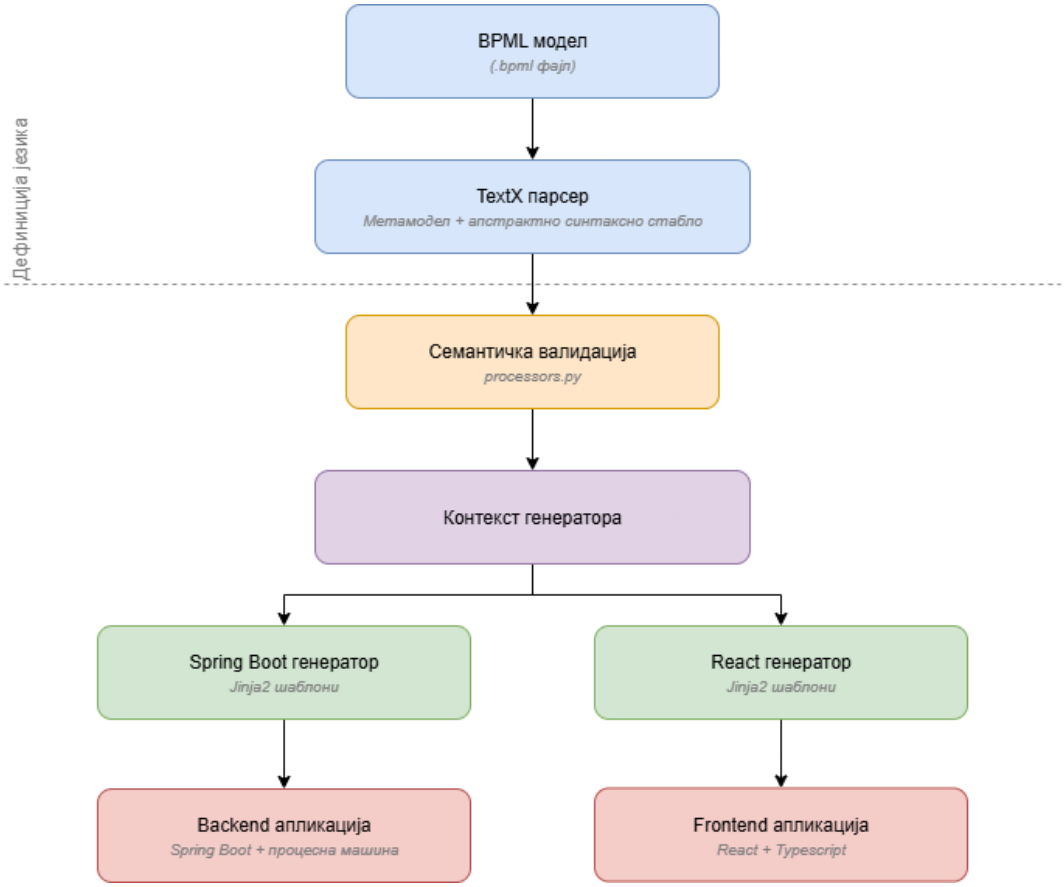
3.1 Архитектура система

Систем је организован у три логичка слоја који одражавају фазе обраде BPML модела: слој дефиниције језика, слој семантичке обраде и слој генерације кода.

Слој дефиниције језика обухвата TextX граматику (`bpml.tx`), којом се дефинише синтакса BPML језика. На основу граматике TextX аутоматски генерише метамодел, тј. формалну дефиницију структуре језика и конструише апстрактно синтаксно стабло за сваки улазни модел. У овом слоју се налази и дефиниција типског система (`builtins.py`) која обухвата уграђене типове и класу `DataType` (`custom_model.py`) за обраду сложених типских референци.

Слој семантичке обраде реализован је кроз модул за валидацију модела (`processors.py`). У овом слоју се врше провјере конзистентности и исправности модела прије фазе генерације кода. Семантичке провјере регистроване су као TextX процесори и извршавају се аутоматски након парсирања.

Слој генерације кода реализован је кроз два генератора, Spring Boot backend генератор и React frontend генератор. Оба генератора заснована су на Jinja2 шаблонском механизму. Детаљан опис механизма генерације и структуре генерисаних система дат је у четвртом поглављу.



Слика 1: Архитектурни дијаграм BPML система

Архитектура система, приказана на слици 1, одвија се кроз сљедеће кораке: дефинисање модела у BPML синтакси, парсирање модела употребом TextX алата, формирање метамодела и апстрактног синтаксног стабла, извршавање семантичких провјера, генерисање контекста за шаблонски механизам и генерисање backend и frontend кода. Оваква архитектура омогућава јасно раздвајање доменске спецификације од инфраструктурне имплементације.

3.1.1 Структура пројекта

Имплементација BPML система организована је у модуларну Python структуру:

```
bpml/
├── language/
│      ├── bpml.tx          # TextX граматика
│      ├── processors.py    # Семантичке провјере
│      ├── builtins.py      # Уграђени типови и мапирања
│      └── custom_model.py  # Помоћне класе модела
│
├── generator/
│      ├── springboot/      # Spring Boot генератор и шаблони
│      ├── react/           # React генератор и шаблони
│      └── util/            # Помоћне функције
│            ├── filters.py           # Jinja филтери
│            ├── string_format_util.py # Форматирање имена
│            └── file_util.py         # Рад са фајл системом
│
└── examples/              # Примјери BPML модела
```

Листинг 1: Структура BPML имплементације

Модул `language` садржи граматику, семантичке процесоре и дефиницију типског система. Модул `generator` садржи имплементацију генератора, Jinja шаблоне за објециљне платформе и помоћне функције за мапирање типова, форматирање идентификатора и управљање фајл системом.

Ова организација омогућава изолацију дефиниције језика од генерације кода, једноставно проширење новим циљним платформама (додавањем новог генератора без измјене граматике или семантичког слоја) и независно тестирање сваког слоја.

3.2 Формална граматика језика

Граматика BPML језика дефинисана је у TextX нотацији, која истовремено служи и као формална спецификација синтаксе и као основа за аутоматско генерисање метамодела и парсера.

3.2.1 TextX граматика

Сљедећи листинг приказује комплетну TextX граматику BPML језика:

```
BusinessProcessModel:
    project_info=ProjectInfo
    processes+=Process
;

ProjectInfo:
    'project' '{'
        'name' ':' projectName=ID
        ('description' ':' description=STRING)?
        ('version' ':' version=STRING)?
```

```
        ('author' ':' author=STRING)?
    '}'
;

Process:
    'process' name=ID '{'
        entities+=Entity
        roles+=Role
        states+=State
        tasks+=Task
        transitions+=Transition
    '}'
;

Entity:
    'entity' name=ID '{'
        attributes*=Attribute
    '}'
;

Attribute:
    name=ID ':' type=DataType
;

DataType:
    'int' | 'float' | 'string' | 'boolean' | Enum
;

Enum:
    'enum' '{' values+=ID[','] '}'
;

Role:
    'role' name=ID
    ('supervises' supervised_roles+=[Role][','])?
;

State:
    'state' name=ID
;

Transition:
    'transition' name=ID
    'from' from_state=[State]
    'to' to_state=[State]
    'by' role=[Role]
```

```
;

Task:
    'task' name=ID
    'in' state=[State]
    (('by' role=[Role]) | auto?='auto')
    ('affects' entities+=[Entity][','])?
    ('depends_on' dependencies+=[Task][','])?
;

Comment:
    /\/\/.*$/
;
```

Листинг 2: Комплетна TextX граматика BPML језика

Граматика дефинише хијерархијску структуру модела: `BusinessProcessModel` као кориј енски елемент садржи метаподатке о пројекту (`ProjectInfo`) и листу процеса (`Process`). Сваки процес обухвата ентитете, улоге, стања, задатке и транзиције.

Кључна синтаксна карактеристика граматике јесте употреба TextX механизма референтног повезивања. Нотација `[State]` (у угластим заградама) означава референцу на претходно дефинисан елемент типа `State`, а не нову дефиницију. На примјер, `from_state=[State]` у правилу за транзицију значи да се наводи назив постојећег стања, а TextX аутоматски резолвира ту референцу у одговарајући објекат модела. Овај механизам омогућава формално повезивање елемената и представља основу за семантичку валидацију.

Нотација `auto?='auto'` у правилу за задатак представља опциони булови атрибут: уколико је кључна ријеч `auto` присутна, атрибут добија вриједност `True`, а у супротном `False`. Ова конструкција, заједно са алтернацијом `('by' role=[Role]) | auto?='auto'`, обезбјеђује да задатак мора бити или ручни (додијељен улози) или аутоматски.

3.2.2 EBNF приказ

Ради формалне потпуности, у наставку је дат EBNF приказ граматике, који је независан од TextX нотације и прати стандардну формалну спецификацију:

```
(* Лексички симболи: *)
ID      = /* идентификатор */ ;
STRING  = /* ниска под наводницима */ ;

(* Почетни симбол: *)
BusinessProcessModel
  = ProjectInfo , { Process } ;
```

```
ProjectInfo
  = "project" , "{" ,
      "name" , ":" , ID ,
      [ "description" , ":" , STRING ] ,
      [ "version" , ":" , STRING ] ,
      [ "author" , ":" , STRING ] ,
    "}" ;

Process
  = "process" , ID , "{" ,
      Entity ,
      Role ,
      State ,
      Task ,
      Transition ,
    "}" ;

Entity
  = "entity" , ID , "{" , { Attribute } , "}" ;

Attribute = ID , ":" , DataType ;

DataType
  = "int" | "float" | "string" | "boolean" | Enum ;

Enum = "enum" , "{" , ID , { "," , ID } , "}" ;

Role
  = "role" , ID ,
    [ "supervises" , ID , { "," , ID } ] ;

State = "state" , ID ;

Transition
  = "transition" , ID ,
    "from" , ID , "to" , ID , "by" , ID ;

Task
  = "task" , ID , "in" , ID ,
    ( ("by" , ID) | "auto" ) ,
    [ "affects" , ID , { "," , ID } ] ,
    [ "depends_on" , ID , { "," , ID } ] ;

Comment
  = "//" , { ? било који знак осим краја реда ? } ;
```

Листинг 3: Формални EBNF приказ синтаксе BPML језика

У EBNF приказу, референце на елементе (нпр. `state=[State]` у TextX нотацији) поједностављено су представљене као `ID`, будући да се њихова исправност обезбјеђује семантичком валидацијом, а не синтаксном анализом.

3.2.3 Кључне ријечи језика

У табели 1 дат је систематски преглед свих кључних ријечи BPML језика, груписаних по категоријама, са описом њихове семантичке улоге.

Табела 1: Систематизација кључних ријечи BPML језика

Кључна ријеч	**Категорија**	**Опис семантике**
project	Метаподаци	Дефинише основне информације о пројекту.
name	Метаподаци	Назив пројекта.
description	Метаподаци	Текстуални опис пројекта.
version	Метаподаци	Верзија модела.
author	Метаподаци	Аутор модела.
process	Структура	Дефинише нови пословни процес.
entity	Доменски модел	Дефинише пословни ентитет са атрибутима.
enum	Типски систем	Дефинише набројив тип.
int	Типски систем	Цјелобројни тип података.
float	Типски систем	Децимални број.
string	Типски систем	Текстуални тип.
boolean	Типски систем	Логичка вриједност (true/false).
role	Контрола приступа	Дефинише учесника у процесу.
supervises	Контрола приступа	Дефинише хијерархију надређености улога.
state	Процесни модел	Представља стање у животном циклусу процеса.
task	Извршавање	Дефинише оперативни корак унутар стања.
in	Извршавање	Повезује задатак са стањем.
by	Извршавање	Означава улогу одговорну за задатак или транзицију.

Кључна ријеч	Категорија	Опис семантике
auto	Извршавање	Аутоматски задатак без корисничке интервенције.
depends_on	Извршавање	Дефинише зависност између задатака.
affects	Извршавање	Означава ентитете на које задатак утиче.
transition	Процесни модел	Дефинише прелаз из једног стања у друго.
from	Процесни модел	Полазно стање транзиције.
to	Процесни модел	Циљно стање транзиције.

Систематизација приказана у табели 1 показује да BPML језик комбинује декларативно моделовање процеса, доменских ентитета и контроле приступа унутар јединствене формалне синтаксе. Укупно 23 кључне ријечи покривају четири семантичке категорије: метаподатке, доменски модел, процесни модел и извршавање.

3.3 Структура BPML модела

BPML модел састоји се из блока пројекта и једног или више процеса. Блок пројекта (`ProjectInfo`) садржи метаподатке: обавезни назив пројекта и опционе атрибуте (опис, верзија, аутор). Процес представља централну јединицу спецификације и садржи све елементе потребне за описивање животног циклуса пословног процеса: ентитете, улоге, стања, задатке и транзиције.

3.3.1 Ентитети и типски систем

Ентитети представљају доменске објекте који се чувају у бази података и над којима се извршавају операције током процеса. Сваки ентитет садржи листу атрибута, при чему сваки атрибут има назив и тип.

Типски систем BPML језика подржава четири основна типа: `string` за текстуалне вриједности, `int` за цјелобројне вриједности, `float` за децималне вриједности и `boolean` за логичке вриједности. Поред основних типова, подржани су и набројиви типови (`enum`), који се дефинишу инлајн унутар атрибута ентитета навођењем могућих вриједности. На примјер, атрибут `category: enum { SUPPLIES, SERVICES, EQUIPMENT, TRAVEL }` дефинише набројив тип са четири вриједности.

Типски систем је проширен механизмом мапирања на циљне платформе. У модулу `builtins.py` дефинисане су мапе које преводе доменске типове у одговарајуће типове Java и TypeScript језика. Ова апстракција омогућава да доменски модел остане независан од имплементационих детаља циљних технологија; исти BPML тип `int` преводи се у `Integer` у Java контексту и у `number` у TypeScript контексту.

3.3.2 Улоге и контрола приступа

Улоге дефинишу актере процеса и омогућавају моделовање хијерархије надређености путем кључне ријечи `supervises`. На примјер, декларација `role CFO supervises Manager, FinanceTeam` дефинише улогу CFO која надгледа улоге Manager и FinanceTeam. Ова хијерархија користи се за контролу приступа током извршавања задатака и транзиција у генерисаном систему.

Семантичка правила обезбјеђују јединственост назива улога унутар процеса, забрану самонадгледања (улога не може надгледати саму себе) и исправност референци. Све надгледане улоге морају бити претходно дефинисане. Овај посљедњи аспект заслужује посебну пажњу: пошто TextX граматика захтијева да се елемент дефинише прије него што се на њега референцира, а `supervises` референцира друге улоге, кружне зависности у хијерархији улога се структурно спречавају. Улога A не може надгледати улогу B која надгледа улогу A, јер би у том случају једна од њих морала бити дефинисана прије друге, а истовремено референцирати елемент који још није дефинисан. Овим механизмом обезбијеђена је ацикличност хијерархије без потребе за експлицитном провјером циклуса.

3.3.3 Процесни елементи: стања, задаци и транзиције

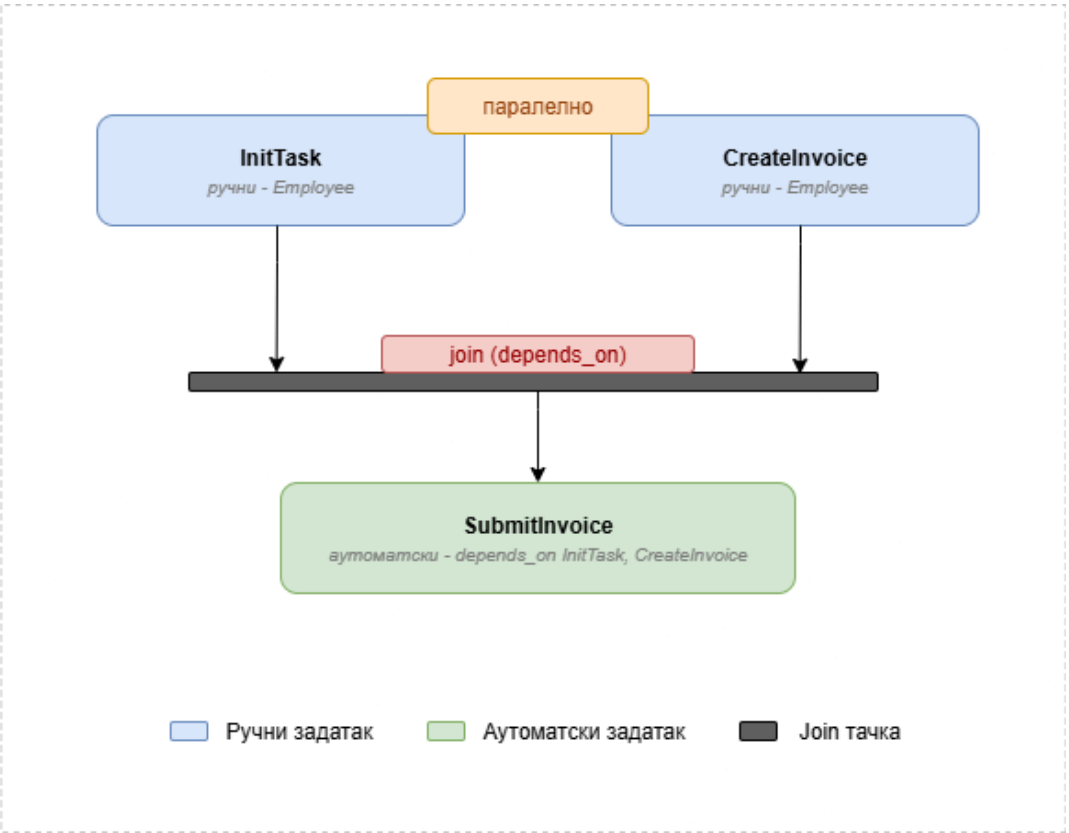
Стања, задаци и транзиције чине процесни модел, срж BPML језика. Заједно, ови елементи описују комплетан животни циклус пословног процеса.

Стања (дефинисана кључном ријечју `state`) представљају фазе животног циклуса процеса. Прво дефинисано стање у процесу имплицитно се третира као почетно стање, а стања из којих не постоје излазне транзиције третирају се као завршна стања. Семантичке провјере обезбјеђују јединственост назива стања унутар процеса.

Транзиције дефинишу контролисане прелазе између стања. Свака транзиција одређена је називом, полазним стањем (`from`), одредишним стањем (`to`) и улогом која је овлашћена за извршавање транзиције (`by`). Семантичке провјере обезбјеђују да полазно и одредишно стање постоје, да нису идентична (забрана самотранзиција) и да наведена улога постоји у процесу. Из једног стања може постојати више излазних транзиција, чиме се омогућава гранање процеса. На примјер, из стања прегледа процес може прећи у стање одобрења или стање одбијања.

Задаци представљају јединице рада унутар одређеног стања. Сваки задатак везан је за тачно једно стање (кључна ријеч `in`) и може бити ручни (додијељен улози путем `by`) или аутоматски (означен кључном ријечју `auto`). Синтакса језика намеће правило да задатак мора бити или ручни или аутоматски, али не обоје истовремено. Задатак може опционо навести ентитете на које утиче (`affects`) и зависности од других задатака (`depends_on`).

Механизам зависности задатака представља кључни елемент за моделовање паралелних токова. Уколико више задатака унутар истог стања нема међусобних зависности, они се могу извршавати паралелно. Ово представља имплицитну fork семантику. Уколико задатак зависи од једног или више претходних задатака, он представља join тачку и може се извршити тек након завршетка свих задатака од којих зависи. Сљедећи дијаграм илуструје ову структуру:



Слика 2: Илустрација fork/join структуре задатака

На примјер, у стању `DraftState` процеса одобравања фактура задаци `InitTask` и `CreateInvoice` немају међусобне зависности и могу се извршавати паралелно. Задатак `SubmitInvoice` дефинисан је са `depends_on InitTask, CreateInvoice`, што значи да ће бити извршен тек након завршетка оба претходна задатка. Будући да је `SubmitInvoice` означен као `auto`, процесна машина ће га аутоматски завршити чим се зависности испуне.

Семантичке провјере обезбјеђују да зависности референцирају постојеће задатке и да задатак не зависи од самог себе. Као и код хијерархије улога, циркуларне зависности између задатака структурно су спријечене: пошто TextX резолвира референце на

основу редосљеда дефинисања, задатак може зависити само од претходно дефинисаних задатака.

3.4 Семантичка валидација

Након синтаксне анализе, над парсираним моделом извршава се статичка семантичка провјера. Ова фаза представља критичну баријеру између спецификације и генерације кода. Она спречава да модел са логичким грешкама буде трансформисан у некоректан изворни код.

Семантичка валидација имплементирана је у модулу `processors.py` и регистрована као TextX процесор који се аутоматски извршава након парсирања. Главна функција `semantic_check` покреће каскаду провјера организованих по типу елемента.

Провјере су структуриране хијерархијски: прво се валидирају метаподаци пројекта (присуство назива), затим се за сваки процес провјерава јединственост назива, а потом се валидира унутрашња структура процеса — ентитети, улоге, стања, задаци и транзиције. Сљедећи исјечак приказује структуру валидационог процеса:

```
def semantic_check(metamodel, model):
    _validate_project_info(metamodel.project_info)
    _validate_processes(metamodel.processes)

def _validate_process_structure(process):
    entity_names = _validate_entities(
        process.entities, process.name)
    role_names = _validate_roles(
        process.roles, process.name)
    state_names = _validate_states(
        process.states, process.name)
    _validate_tasks(process.tasks,
        state_names, role_names,
        entity_names, process.name)
    _validate_transitions(process.transitions,
        state_names, role_names, process.name)
```

Листинг 4: Структура семантичке валидације

Свака функција валидације враћа скуп валидних назива, који се затим прослијеђује наредним функцијама. На примјер, `_validate_entities` враћа скуп назива ентитета, који се користи у `_validate_tasks` за провјеру валидности `affects` референци. Ова каскадна структура обезбјеђује да се грешке детектују у најранијој могућој фази.

Укупно, семантички слој провјерава: јединственост назива свих елемената (процеси, ентитети, улоге, стања, задаци, транзиције), исправност свих референци (стања у задацима и транзицијама, улоге у задацима и транзицијама, ентитети у задацима, зависности задатака), конзистентност хијерархије улога (забрана самонадгледања, забрана

референци на непостојеће улоге), конзистентност транзиција (забрана самотранзиција, провјера полазних и одредишних стања), конзистентност задатака (ексклузивитет ручни/аутоматски, забрана самозависности) и исправност зависности задатака.

У случају детекције грешке, систем подиже `TextXSemanticError` са описном поруком која укључује назив проблематичног елемента и назив процеса у којем се грешка налази. Конкретни примјери грешака и одговарајућих порука демонстрирани су у петом поглављу кроз студију случаја.

3.5 Дизајнерске одлуке

Током развоја BPML језика донесено је неколико кључних дизајнерских одлука које су обликовале коначну синтаксу и семантику језика.

3.5.1 Имплицитни паралелизам

Најзначајнија дизајнерска одлука тиче се моделовања паралелних токова. Разматран је приступ са експлицитном синтаксом за `fork` и `branch` конструкције:

```
state OnboardingState {
    fork {
        branch ITSetup:
            SetupWorkstation, AssignEquipment
        branch Training:
            ScheduleOrientation, ConductTraining
    }
}
```

Листинг 5: Разматрана алтернатива — експлицитна fork/branch синтакса

Овакав приступ чини паралелизам изричитим и ближим формалним workflow моделима. Међутим, анализа употребљивости показала је да таква синтакса уноси редундансу, повећава комплексност језика и оптерећује корисника додатним декларацијама. Усвојено је једноставније рјешење у којем је паралелизам имплицитан: задаци који немају међусобне зависности могу се извршавати паралелно, док се синхронизација остварује путем `depends_on` механизма (join тачака). Овим приступом задржана је минималистичка синтакса, а семантика паралелизма остаје јасно дефинисана.

3.5.2 Избор TextX алата

Одлука о употреби TextX алата [2] за имплементацију BPML језика донесена је на основу неколико фактора. TextX обједињује дефиницију граматике и метамодела у јединственој спецификацији, чиме се елиминише потреба за одвојеном дефиницијом апстрактне синтаксе. Додатно, TextX је имплементиран у Python-у, што омогућава природну интеграцију са осталим дијеловима BPML система (семантички процесори, генератори, помоћне функције). Механизам референтног повезивања (`[ClassName]`)

обезбјеђује аутоматско резолвирање референци, а подршка за регистрацију генератора путем `@generator` декоратора олакшава интеграцију са генеративним слојем.

Алтернативе попут ANTLR-а нуде богатији скуп могућности за парсирање сложених језика, али захтијевају засебну дефиницију апстрактне синтаксе и не обезбјеђују уграђену подршку за метамоделовање. За потребе BPML језика, чија граматика има релативно једноставну структуру, TextX представља оптималан избор.

3.5.3 Избор Jinja2 шаблонског механизма

За генерацију кода одабран је Jinja2 шаблонски механизам, интегрисан са TextX екосистемом путем библиотеке `textx-jinja`. Овај избор омогућава раздвајање структуре генерисаног кода (дефинисане у шаблонима) од логике трансформације (дефинисане у Python генераторима). Jinja2 подржава наслијеђивање шаблона, филтере, макрое и контролне структуре, што је довољно за генерацију сложених Java и TypeScript фајлова. Детаљни механизам генерације описан је у четвртом поглављу.

3.6 Алатска подршка: VSCode екстензија

Поред саме дефиниције језика и генератора кода, за BPML језик развијена је екстензија за Visual Studio Code едитор, која обезбјеђује подршку за писање `.bpml` фајлова. Екстензија представља допуну текстуалној природи језика. Иако BPML не располаже графичким едитором, VSCode екстензија значајно олакшава рад са текстуалном синтаксом и смањује когнитивно оптерећење приликом моделовања процеса.

3.6.1 Синтаксно бојење

Синтаксно бојење реализовано је путем TextMate граматике (`bpml.tmLanguage.json`), која представља стандардни механизам за дефинисање синтаксних правила у VSCode екосистему. Граматика класификује лексичке елементе BPML језика у неколико категорија, од којих свака добија одговарајућу визуелну репрезентацију у едитору.

Примарне кључне ријечи (`project`, `process`, `entity`, `role`, `state`, `task`, `transition`) означене су као `keyword.control` и визуелно се истичу као контролне структуре језика. Секундарне кључне ријечи (`name`, `description`, `supervises`, `from`, `to`, `by`, `in`, `affects`, `depends_on`, `auto`) означене су као `keyword.other`. Типови података (`int`, `float`, `string`, `boolean`, `enum`) означени су као `storage.type`. Додатно, граматика обезбјеђује бојење ниски под наводницима, коментара (`//`) и разликовање идентификатора. Имена која почињу великим словом (типично називи ентитета, процеса и стања) визуелно се разликују од имена која почињу малим словом (типично називи атрибута).

На слици 3 приказано је синтаксно бојење BPML фајла у VSCode едитору, што илуструје различите категорије лексичких елемената и њихову визуелну репрезентацију.

```
example.bpml M ×
bpml > examples > example.bpml
You, last month | 1 author (You)
// Invoice Approval Business Process        You, 5 months ago • Simplify the language …

project {
    name: InvoiceApprovalSystem
    description: "Automated invoice approval workflow with multi-level authorization"
    version: "1.0.0"
    author: "BPML Team"
}

process InvoiceApproval {
    // Define the domain entities
    entity Invoice {
        invoiceNumber: string
        vendor: string
        amount: float
        description: string
        invoiceDate: string
        dueDate: string
        category: enum { SUPPLIES, SERVICES, EQUIPMENT, TRAVEL }
        attachmentUrl: string
    }

    // Define roles in the process with hierarchy
    role Employee
    role Manager supervises Employee
    role FinanceTeam
    role CFO supervises Manager, FinanceTeam

    // Define process states
    state DraftState
    state SubmittedForApprovalState

    // Define workflow tasks (tasks belong to specific states)
    task InitTask in DraftState by Employee
    task CreateInvoice in DraftState by Employee affects Invoice
    task SubmitInvoice in DraftState auto affects Invoice depends_on InitTask, CreateInvoice
    task ReviewByManager in SubmittedForApprovalState by Manager affects Invoice, ApprovalRecord

    // Define state transitions
    transition InitiateDraftTransition
        from DraftState
        to SubmittedForApprovalState
        by Employee
}
```

Слика 3: Синтаксно бојење BPML фајла у VSCode едитору

Овакво бојење омогућава кориснику тренутну визуелну повратну информацију о структури модела и олакшава уочавање синтаксних грешака.

3.6.2 Исјечци кода

Екстензија обезбјеђује дванаест предефинисаних исјечака кода (енг. *code snippets*) који омогућавају брзо креирање BPML конструкција. Сваки исјечак се активира уносом одговарајућег префикса и притиском на тастер `Tab`, након чега се умеће параметризовани блок кода са означеним мјестима за унос вриједности.

Подржани су исјечци за све кључне елементе језика: `project` за блок метаподатака, `process` за дефиницију процеса, `entity` и `entity-enum` за ентитете, `role` и `role-supervises` за улоге, `state` за стања, `task`, `task-affects`, `task-depends` и `task-auto` за различите варијанте задатака, и `transition` за транзиције. Додатно, исјечак `bpml-template` генерише комплетну структуру BPML фајла са свим основним елементима, чиме се значајно убрзава почетно креирање модела.

Сљедећи примјер илуструје исјечак за дефиницију транзиције:

```
"Transition Definition": {
    "prefix": "transition",
    "body": [
        "transition ${1:TransitionName}",
        "\tfrom ${2:FromState}",
        "\tto ${3:ToState}",
        "\tby ${4:Role}"
    ],
    "description": "Create a transition definition"
}
```

Листинг 6: VSCode исјечак за креирање транзиције

Означена мјеста `${1:TransitionName}`, `${2:FromState}` итд. представљају параметре које корисник попуњава секвенцијалним притиском на `Tab`. Подразумијеване вриједности (нпр. `TransitionName`, `FromState`) служе као водич кориснику.

3.6.3 Конфигурација језика

Конфигурација језика (`language-configuration.json`) дефинише додатне функционалности едитора специфичне за BPML синтаксу. Подржано је аутоматско затварање заграда и наводника, нпр. када корисник откуца отворену витичасту заграду `{`, едитор аутоматски умеће затворену `}`. Аналогно се понашају угласте заграде, обле заграде и двоструки наводници. Подржано је и склапање блокова кода (енг. *code folding*): блокови ограничени витичастим заградама (нпр. тијело процеса, ентитета) могу се склопити ради прегледности. Коментари се уносе стандардним пречицама за линијске коментаре (`//`).

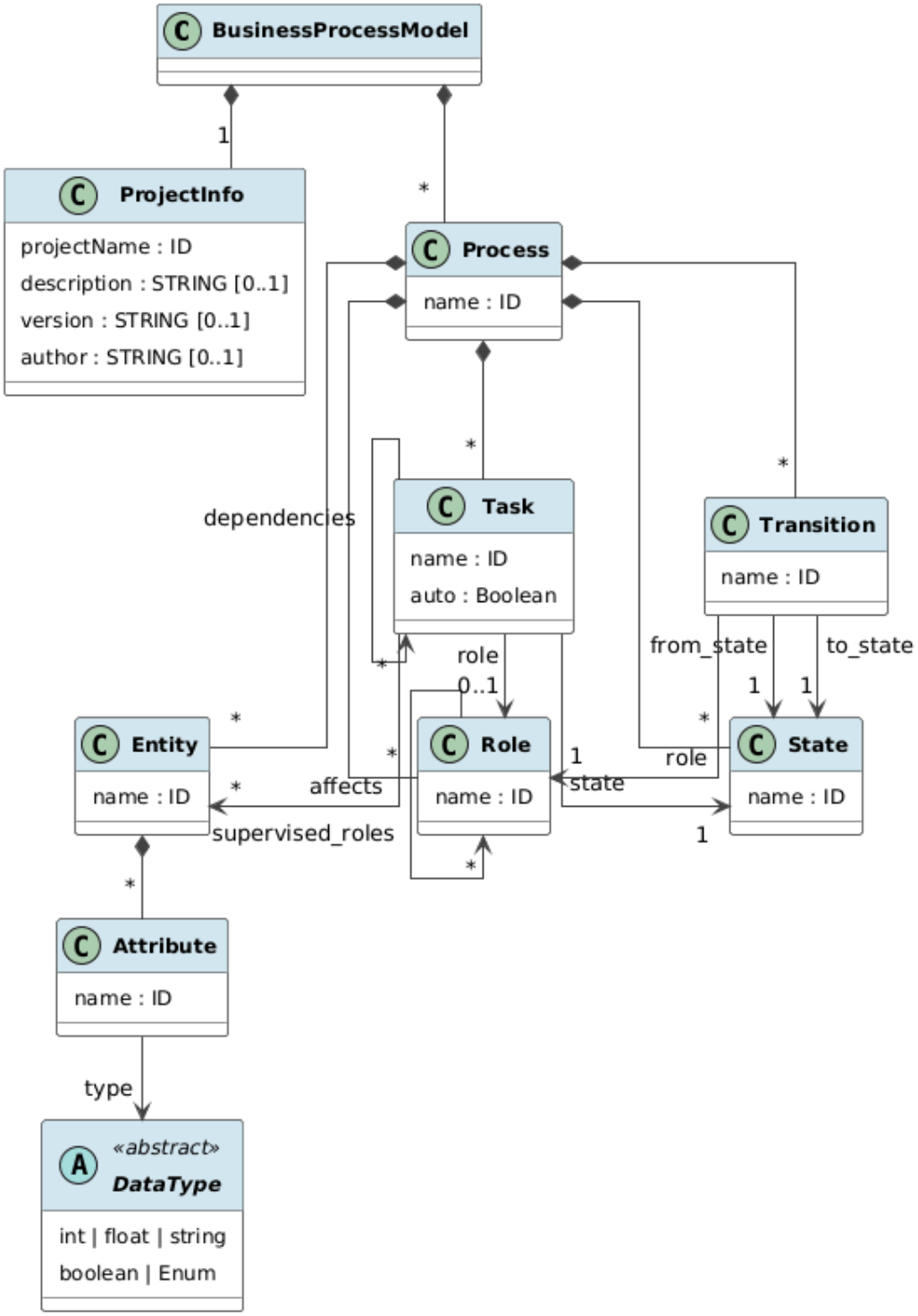
3.6.4 Инсталација и употреба

Екстензија се може инсталирати копирањем у директоријум VSCode екстензија или паковањем у `.vsix` формат путем `vsce` алата. Након инсталације, VSCode аутоматски препознаје фајлове са `.bpml` екстензијом и активира синтаксно бојење, исјечке и конфигурацију језика.

Развој екстензије организован је тако да се синтаксно бојење, исјечци и конфигурација језика могу независно проширивати. Додавање нових кључних ријечи у граматику или нових исјечака не захтијева измјене у осталим дијеловима екстензије.

3.7 Метамодел и односи елемената

Метамодел BPML (слика 4) језика дефинише формалне односе између елемената и представља основу за семантичку анализу и изградњу контекста за генерацију кода.



Слика 4: UML дијаграм метамодела BPML језика

На највишем нивоу, `BusinessProcessModel` садржи један `ProjectInfo` објекат и листу `Process` објеката. Сваки процес агрегира листе ентитета, улога, стања, задатака и транзиција. Односи између елемената унутар процеса имају карактер референци: задатак референцира стање (обавезно), улогу (опционо), ентитете (опционо) и друге задатке као зависности (опционо). Транзиција референцира полазно стање, одредишно стање и улогу. Улога може референцирати друге улоге кроз механизам надгледања.

Ови односи формирају граф зависности унутар процеса. Чворови графа су елементи модела (стања, задаци, транзиције, улоге, ентитети), а гране представљају референце између њих. Семантичка валидација описана у претходној секцији обезбјеђује конзистентност овог графа. Све гране морају показивати на постојеће чворове, не смију постојати циклуси у хијерархији улога и зависностима задатака, а одређена структурна правила (ексклузивитет ручни/аутоматски, забрана самотранзиција) морају бити задовољена.

Метамодел директно се пресликава у контекст генерације кода: за сваки ентитет генеришу се одговарајуће класе, за сваку улогу генерише се елемент контроле приступа, а стања и транзиције формирају статичку конфигурацију процесне машине у генерисаном backend систему.

Глава 4

Генерисање кода

На основу спецификације дефинисане у BPML моделу, систем омогућава аутоматску генерацију комплетне full-stack апликације. Генерисани систем обухвата backend апликацију засновану на Spring Boot платформи и frontend апликацију засновану на React оквиру са TypeScript језиком. Основна идеја овог приступа јесте да се доменска логика дефинише декларативно у DSL-у, док се инфраструктурни код генерише аутоматски, чиме се минимизује ручно писање понављајућих дијелова система.

Генерација кода представља централну фазу у животном циклусу BPML система. Она повезује апстрактну спецификацију пословног процеса са конкретном извршивом имплементацијом, чиме се остварује директна веза између модела и софтверског производа. Овај приступ је у складу са принципима моделом вођеног развоја [5] и генеративног програмирања [10].

У наставку поглавља описан је механизам генерације кода, архитектура и структура генерисаног backend система, архитектура и компоненте генерисаног frontend система, њихова међусобна интеграција, као и предности оваквог аутоматизованог приступа.

4.1 Механизам генерације кода

Генерација кода у BPML систему заснива се на шаблонском приступу, при чему се структура и садржај генерисаних фајлова дефинишу путем параметризованих шаблона. За реализацију овог механизма користи се Jinja2 шаблонски језик, интегрисан са TextX екосистемом путем библиотеке `textx-jinja`. Овакав приступ омогућава раздвајање генерисане структуре кода од логике трансформације модела, чиме се поједностављује одржавање и проширење генератора.

4.1.1 Контекст генерације

Прије покретања шаблонског механизма, из BPML модела се извлачи контекст, структура података која садржи све информације потребне за генерисање кода. Контекст се формира на основу парсираног модела и обухвата:

- назив пројекта и метаподатке,
- листу свих процеса дефинисаних у моделу,
- листу ентитета са њиховим атрибутима,
- стања, задатке и транзиције за сваки процес,
- улоге и њихову хијерархију.

Сљедећи исјечак приказује функцију за формирање контекста у Spring Boot генератору:

```
def get_context(model):
    project_name = model.project_info.projectName
    all_entities = []
    all_entity_names = []
    for process in model.processes:
        all_entities.extend(process.entities)
        all_entity_names.extend(
            [e.name.lower() for e in process.entities]
        )
    context = {
        'group_name': 'com.bpml',
        'app_name': project_name,
        'app_name_lower': project_name.lower(),
        'app_name_cap': capitalize_str(project_name),
        'project_info': model.project_info,
        'processes': model.processes,
        'entities': all_entities,
        'entity_names': all_entity_names
    }
    return context
```

Листинг 7: Формирање контекста за генерацију backend система

Контекст се прослијеђује Jinja шаблонима, гдје се динамички замјењују параметри конкретним вриједностима из модела. На тај начин, исти шаблон може произвести различите фајлове за различите ентитете или процесе.

4.1.2 Jinja шаблони и филтери

Jinja2 шаблони представљају параметризоване текстуалне фајлове чији се дијелови динамички попуњавају на основу контекста. Шаблони користе синтаксу `{{ ... }}` за интерполацију вриједности и `{% ... %}` за контролне структуре (услове, петље).

Поред основних механизама интерполације, систем дефинише прилагођене филтере који се користе унутар шаблона за трансформацију података. Филтери обухватају двије категорије функционалности.

Прву категорију чине филтери за мапирање типова, који преводе доменске типове BPML језика у одговарајуће типове циљних платформи. Сљедећи исјечак приказује дефиницију мапирања типова:

```
# Java type mappings
data_type_java_mapper = {
    "int": "Integer",
    "string": "String",
    "float": "Float",
    "boolean": "Boolean",
}

# TypeScript type mappings
data_type_typescript_mapper = {
    "int": "number",
    "string": "string",
    "float": "number",
    "boolean": "boolean",
}
```

Листинг 8: Мапирање BPML типова на Java и TypeScript типове

Другу категорију чине филтери за форматирање идентификатора, који обезбјеђују усклађеност именовања са конвенцијама циљних платформи. Систем подржава конверзију између различитих нотација: `CamelCase` за Java класе, `camelCase` за JavaScript варијабле, `snake_case` за Python идентификаторе, `dash-case` за URL путање и `UPPER_CASE` за константе. На примјер, идентификатор `InvoiceApproval` из BPML модела конвертује се у `invoice-approval` за URL путање, `invoiceApproval` за TypeScript варијабле и `INVOICE_APPROVAL` за Java константе.

Оваква апстракција омогућава да се у BPML моделу користи јединствена конвенција именовања, док генератори аутоматски примјењују одговарајуће конвенције за сваку циљну платформу.

4.1.3 Процес генерације

TextX екосистем омогућава регистрацију генератора путем декоратора `@generator`, чиме се успоставља формална веза између изворног језика и циљне платформе. Сваки генератор прима метамодел, модел, излазну путању и додатне параметре, а затим покреће шаблонски механизам за генерисање циљних фајлова.

Процес генерације одвија се у сљедећим корацима:

1. Парсирање BPML модела и формирање апстрактног синтаксног стабла.
2. Извршавање семантичких провјера.
3. Формирање контекста генерације на основу модела.
4. Регистрација филтера за шаблонски механизам.
5. Итерација кроз елементе модела (ентитети, процеси) и генерисање одговарајућих фајлова путем Jinja шаблона.

За сваки ентитет дефинисан у моделу, контекст се проширује специфичним подацима о том ентитету (назив, атрибути, типови), а затим се покреће генерисање свих повезаних фајлова, од модела до контролера. Аналогно, за сваки процес контекст се проширује подацима о стањима, задацима, транзицијама и улогама, након чега се генеришу фајлови процесног runtime система.

Сљедећи исјечак илуструје генерисање фајлова за ентитете у backend генератору:

```
def generate_entity_files(context, filters,
                          main_folder_path, model, overwrite):
    template = os.path.join(THIS_FOLDER,
                            'template/content_structure')
    for entity in all_entities:
        context['entity'] = entity
        context['entity_name'] = entity.name
        context['entity_name_cap'] = capitalize_str(entity.name)
        context['attributes'] = entity.attributes

        for attribute in context['attributes']:
            if is_enum_type(attribute.type):
                context['enum_values'] = \
                    get_enum_values(attribute.type)
                enum_template = os.path.join(
                    THIS_FOLDER, 'template/enum_files')
                textx_jinja_generator(enum_template,
                    main_folder_path, context,
                    overwrite, filters=filters)

        textx_jinja_generator(template, main_folder_path,
            context, overwrite, filters=filters)
```

Листинг 9: Генерисање backend фајлова за сваки ентитет

На овај начин, додавање новог ентитета у BPML модел аутоматски резултира генерисањем комплетног скупа фајлова за тај ентитет, без потребе за ручном интервенцијом.

4.1.4 Организација шаблона

Шаблони су организовани у двије хијерархије, једну за Spring Boot backend и другу за React frontend. Свака хијерархија садржи подгрупе шаблона: структуру пројекта, фајлове за ентитете, фајлове процесног runtime-а, конфигурационе фајлове и помоћне компоненте.

Називи шаблонских фајлова параметризовани су тако да се имена генерисаних фајлова аутоматски формирају на основу назива ентитета или процеса. На примјер, шаблон `__entity_name__List_tsx.jinja` производи фајл `InvoiceList.tsx` када се примијени на ентитет `Invoice`.

4.2 Генерисање backend система

На основу BPML спецификације, Spring Boot генератор производи потпуно функционалан серверски систем. Генерисана апликација прати слојевиту архитектуру и обухвата доменски модел, слој перзистенције, сервисни слој, REST интерфејс, процесну машину и механизме контроле приступа.

4.2.1 Доменски модел и перзистенција

За сваки ентитет дефинисан у BPML моделу генерише се JPA (Java Persistence API) ентитет означен одговарајућим анотацијама. Атрибути ентитета мапирају се на Java типове у складу са типским мапирањем дефинисаним у слоју уграђених типова. Генерисани ентитет укључује аутоматски генерисани примарни кључ, мапирање набројивих типова на Java `enum` класе и поље за повезивање са инстанцом процеса.

Набројиви типови дефинисани у BPML моделу (кључна ријеч `enum`) генеришу се као засебне Java enum класе. На примјер, атрибут `category: enum { SUPPLIES, SERVICES, EQUIPMENT, TRAVEL }` из BPML спецификације резултира генерисањем одговарајуће Java enum класе са наведеним вриједностима.

За сваки ентитет генерише се Spring Data репозиторијум који омогућава основне CRUD операције и аутоматску имплементацију упита. Генерисани систем подржава H2 базу података за развојно окружење и PostgreSQL за продукциони контекст.

Перзистенција процесних инстанци и задатака реализована је преко засебних ентитета: `ProcessInstance` и `TaskInstance`. Процесна инстанца чува тренутно стање процеса, статус, временске ознаке, идентификатор корисника који је покренуо процес и процесне варијабле серијализоване у JSON формату. Инстанца задатка чува назив задатка, додијељену улогу, статус, листу зависности и листу ентитета на које задатак утиче. Додатно, ентитет `TransitionHistory` биљежи историју извршених транзиција, укључујући информације о кориснику, улози и временским ознакама.

Генерисани систем такође укључује DTO (Data Transfer Object) класе и мапере који обезбјеђују раздвајање интерне репрезентације података од интерфејса према клијентима. Овим приступом се спречава директна изложеност интерне структуре ентитета кроз REST интерфејс.

4.2.2 Сервисни слој

Сервисни слој представља апстракцију између контролера и слоја перзистенције. За сваки ентитет генерише се сервисна класа која обрађује пословну логику, позива репозиторијуме, управља трансакцијама и врши валидацију захтјева.

Поред ентитетских сервиса, генеришу се и специјализовани сервиси за управљање процесним инстанцама (`ProcessInstanceService`) и задацима (`TaskService`). Сервис за процесне инстанце обезбјеђује операције читања, филтрирања по статусу и називу

процеса, те пагинацију резултата. Сервис за задатке обезбјеђује преузимање задатака по идентификатору, процесној инстанци, улози и кориснику, као и операције завршетка, поништавања и ескалације задатака.

Сервис за задатке садржи механизам провјере зависности: прије завршетка задатка провјерава се да ли су сви задаци од којих он зависи већ завршени. Уколико зависности нису испуњене, операција завршетка се одбија са одговарајућом поруком о грешци. Ова провјера осигурава да се семантика зависности дефинисана у BPML моделу досљедно примјењује током извршавања.

4.2.3 Процесна машина

Процесна машина представља кључни елемент генерисаног backend система. Она оркестрира извршавање процеса и осигурава да се понашање система у потпуности подудара са спецификацијом дефинисаном у BPML моделу.

Процесна машина имплементирана је као Spring сервис (`ProcessEngine`) који управља цијелим животним циклусом процесних инстанци. Основне одговорности процесне машине обухватају: креирање нових инстанци процеса, праћење тренутног стања, активирање задатака у одговарајућем стању, валидацију и извршавање транзиција, аутоматско завршавање задатака без корисничке интервенције и евидентирање историје извршавања.

Стања, задаци и транзиције дефинисани у BPML моделу претварају се у статичку конфигурацију процесне машине, серијализовану у JSON формату и сачувану у ентитету `ProcessDefinition`. На тај начин, runtime понашање директно одговара спецификацији модела.

Покретање нове инстанце процеса одвија се кроз сљедеће кораке: учитавање дефиниције процеса из базе, одређивање почетног стања, креирање инстанце са статусом `RUNNING`, повезивање ентитета са инстанцом и креирање почетних задатака за иницијално стање. Сљедећи исјечак приказује кључни дио генерисаног кода за извршавање транзиција:

```java
@Transactional
public ProcessInstanceDTO executeTransition(
        Long processInstanceId,
        String transitionName,
        ExecuteTransitionRequest request) {

    ProcessInstance processInstance =
        processInstanceRepository.findById(processInstanceId)
            .orElseThrow(() -> new NotFoundException(
                "Process instance not found"));

    // Валидација статуса процеса
    if (processInstance.getStatus()
            != ProcessInstance.ProcessStatus.RUNNING) {
        throw new IllegalStateException(
            "Cannot execute transition on non-running process");
    }

    // Валидација транзиције
    Map<String, Object> transition =
        findTransition(definitionMap, transitionName);
    String fromState = (String) transition.get("fromState");
    String toState = (String) transition.get("toState");
    String requiredRole = (String) transition.get("role");

    if (!processInstance.getCurrentState().equals(fromState)) {
        throw new IllegalStateException(
            "Invalid transition for current state");
    }

    // Провјера улоге
    if (requiredRole != null
            && !request.getUserRole().equals(requiredRole)) {
        throw new IllegalStateException(
            "User role not authorized for this transition");
    }

    // Промјена стања и креирање нових задатака
    processInstance.setCurrentState(toState);
    processInstanceRepository.save(processInstance);
    createTasksForState(processInstance, toState, definitionMap);

    return mapToProcessInstanceDTO(processInstance);
}
```

Листинг 10: Генерисани Java код за извршавање транзиције

Извршавање транзиције подразумијева четири нивоа валидације: провјеру да процесна инстанца постоји и да је у статусу `RUNNING`, провјеру да се тренутно стање подудара са полазним стањем транзиције, провјеру да корисник посједује одговарајућу улогу и провјеру да су сви задаци у тренутном стању завршени. Тек након успјешне валидације, систем ажурира стање инстанце, биљежи историју транзиције и креира нове задатке за одредишно стање.

Животни циклус процесне инстанце описан је сљедећим статусима:

- `RUNNING` — процес је активан и извршава се,
- `COMPLETED` — процес је успјешно завршен,
- `SUSPENDED` — процес је привремено обустављен,
- `TERMINATED` — процес је прекинут,
- `ERROR` — дошло је до грешке у извршавању.

Задаци имају сопствени животни циклус: задатак се креира у статусу `PENDING`, преузимањем прелази у `IN_PROGRESS`, а завршетком добија статус `COMPLETED`. Задатак може бити и поништен (`CANCELLED`).

Механизам аутоматских задатака заслужује посебну пажњу. Када процесна машина креира задатке за ново стање, за сваки задатак означен као `auto` у BPML моделу провјерава се да ли су испуњене зависности. Уколико задатак нема зависности или су све зависности већ завршене, задатак се аутоматски завршава у тренутку креирања. У супротном, задатак остаје у статусу `PENDING` док се зависности не испуне. Након завршетка било ког задатка, систем провјерава да ли постоје аутоматски задаци чије су зависности сада испуњене и аутоматски их завршава. Ова каскадна провјера обезбјеђује коректно извршавање сложених ланаца зависности.

Провјера завршетка процеса обавља се аутоматски након сваке промјене стања: уколико се процесна инстанца налази у завршном стању и сви задаци су завршени, процес аутоматски прелази у статус `COMPLETED`.

4.2.4 REST интерфејс и обрада грешака

За сваки ентитет дефинисан у BPML моделу генерише се REST контролер који излаже стандардне CRUD операције: креирање (`POST`), читање (`GET`), ажурирање (`PUT`) и брисање (`DELETE`). Поред ентитетских контролера, генеришу се и контролери за управљање процесима који омогућавају покретање процесних инстанци, извршавање транзиција, преузимање активних задатака и увид у историју извршавања.

Процесни REST интерфејс обухвата сљедеће кључне операције: покретање нове инстанце процеса са опционим повезивањем ентитета, преузимање доступних транзиција за тренутно стање, извршавање транзиције уз валидацију улоге, преузимање

задатака филтрираних по улози или кориснику, завршетак задатка уз провјеру зависности и преузимање историје транзиција за инстанцу.

Систем генерише централизовани механизам за обраду изузетака путем `@ControllerAdvice` анотације. Овај механизам обезбјеђује конзистентан формат одговора за све грешке, раздвајање техничких и пословних изузетака и структурирану поруку о грешци која олакшава дијагностику проблема. Дефинисане су двије категорије изузетака: `NotFoundException` за случајеве када тражени ресурс не постоји и `IllegalStateException` за покушаје нелегалних операција, као што су транзиција из погрешног стања или извршавање задатка прије испуњавања зависности.

Генерисана апликација укључује и CORS конфигурацију која омогућава комуникацију frontend апликације са backend-ом током развоја, као и Maven конфигурацију (`pom.xml`) са свим потребним зависностима и `application.properties` фајл са подешавањима базе података, серверског порта и других параметара.

4.3 Генерисање frontend система

На основу BPML модела, React генератор производи комплетну клијентску апликацију засновану на React оквиру и TypeScript језику. Генерисана апликација обезбјеђује кориснички интерфејс за управљање ентитетима, процесним инстанцама и задацима, уз визуелизацију процесног тока.

4.3.1 Типске дефиниције и доменски модел

На основу BPML спецификације генеришу се TypeScript интерфејси који одговарају backend ентитетима. Атрибути ентитета мапирају се из доменских типова у TypeScript типове: `string` остаје `string`, `int` и `float` постају `number`, а `boolean` остаје `boolean`. За набројиве типове генеришу се TypeScript `enum` дефиниције.

Овај приступ омогућава статичку провјеру типова на страни клијента, смањење runtime грешака и синхронизацију типског модела између frontend-а и backend-а. Уколико се промијени структура ентитета у BPML моделу, поновна генерација аутоматски ажурира и TypeScript интерфејсе.

4.3.2 CRUD интерфејси за ентитете

За сваки ентитет дефинисан у BPML моделу генеришу се три React компоненте: листа (`List`), дијалог (`Dialog`) и детаљна страница (`Page`).

Компонента листе приказује табеларни преглед свих инстанци ентитета, при чему се колоне аутоматски генеришу на основу атрибута дефинисаних у BPML моделу. Компонента подржава навигацију на детаљну страницу кликом на ред и отварање дијалога за уређивање.

Сљедећи исјечак приказује дио Jinja шаблона за генерисање компоненте листе:

```
const {{ entity_name }}List: React.FC = () => {
  const [{{ entity_name_lower }}List,
         set{{ entity_name }}List] = useState<{{ entity_name }}[]>([]);

  useEffect(() => { loadData(); }, []);

  const loadData = () => {
    {{ entity_name }}Service.getAll{{ entity_name }}()
      .then((response) => {
        set{{ entity_name }}List(response);
      });
  };

  return (
    <TableContainer component={Paper}>
      <Table>
        <TableHead>
          <TableRow>
            <TableCell>ID</TableCell>
            {% for attr in attributes %}
            <TableCell>{{ attr.name | capitalize_str }}</TableCell>
            {% endfor %}
          </TableRow>
        </TableHead>
        ...
      </Table>
    </TableContainer>
  );
};
```

Листинг 11: Исјечак Jinja шаблона за генерисање React листе ентитета

Овај шаблон илуструје кључну карактеристику генеративног приступа: `{% for attr in attributes %}` петља итерира кроз све атрибуте ентитета дефинисане у BPML моделу и аутоматски генерише одговарајуће колоне табеле. На тај начин, иста шаблонска структура производи различите компоненте за различите ентитете.

Компонента дијалога омогућава креирање и измјену инстанци ентитета путем форме. За сваки атрибут генерише се одговарајуће поље: текстуално поље за `string` тип, нумеричко поље за `int` и `float` типове, и падајући мени за набројиве типове. Форма користи `react-hook-form` библиотеку за управљање стањем и валидацију уноса.

Компонента детаљне странице приказује све атрибуте појединачне инстанце ентитета, омогућава повезивање са процесним инстанцама и пружа акције за уређивање и брисање.

Све компоненте користе Material-UI библиотеку, чиме се обезбјеђује конзистентан и модеран визуелни идентитет генерисаног интерфејса.

4.3.3 Сервисни слој и комуникација са backend-ом

За сваки ентитет генерише се сервис који комуницира са REST интерфејсом backend-а путем HTTP захтјева. Сервисни слој обухвата функције за све CRUD операције (`GET`, `POST`, `PUT`, `DELETE`), обраду одговора и централизовано руковање грешкама. Овај слој омогућава изолацију логике комуникације од UI компоненти, чиме се олакшава тестирање и одржавање генерисаног кода.

Поред ентитетских сервиса, генеришу се и сервиси за комуникацију са процесним API-јем: покретање процеса, извршавање транзиција, преузимање задатака и увид у историју.

4.3.4 Управљање процесима и задацима

Frontend систем омогућава потпуну интеракцију са процесним runtime-ом генерисаног backend-а. За сваки процес дефинисан у BPML моделу генеришу се сљедеће компоненте:

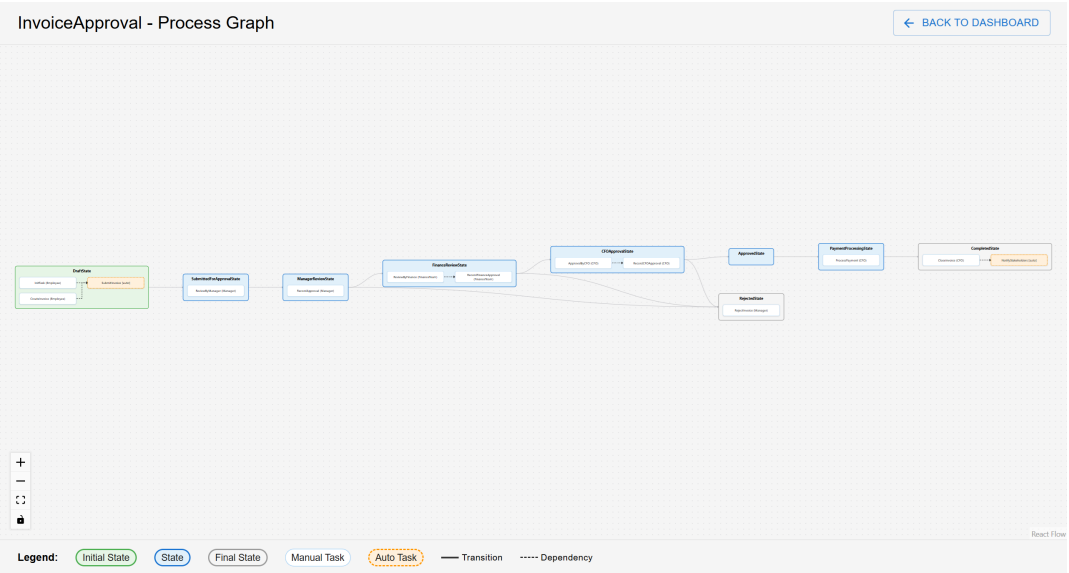
- Dashboard страница — централни преглед процеса са статистикама активних, завршених и обустављених инстанци,
- листа инстанци — табеларни преглед свих инстанци процеса са могућношћу филтрирања по статусу,
- детаљна страница инстанце — приказ тренутног стања, повезаних ентитета, активних задатака и историје транзиција,
- листа задатака — приказ задатака филтрираних по улози корисника.

Задаци представљају централни елемент интеракције корисника са процесом. Генерисане компоненте омогућавају приказ задатака по улогама, преузимање и извршавање задатака, те приказ статуса и зависности. Систем обезбјеђује да кориснику буду доступни искључиво задаци који су валидни у тренутном стању процеса и за његову улогу.

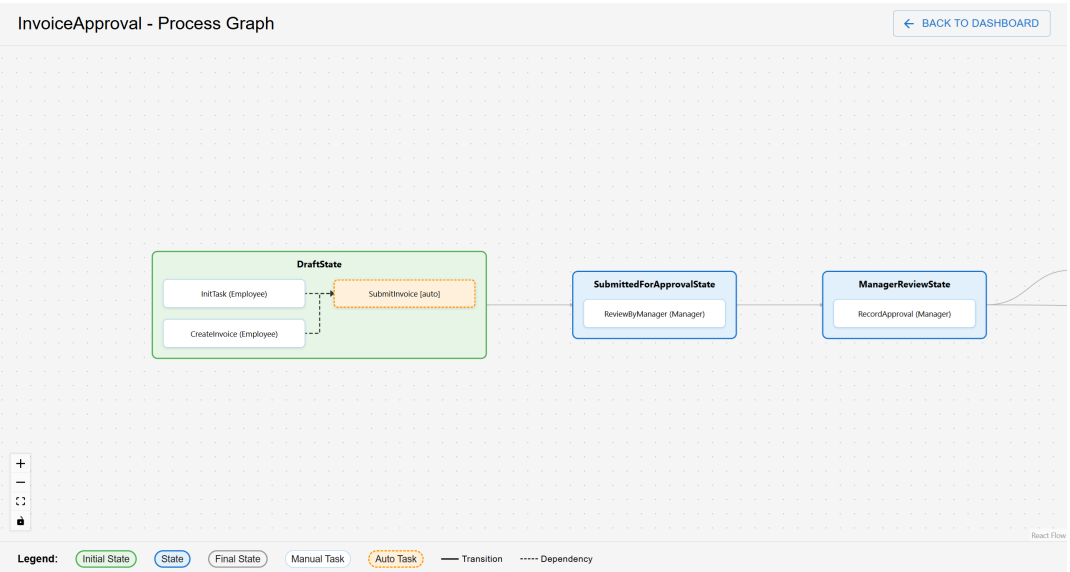
4.3.5 Визуелизација процесног графа

Једна од напредних функционалности генерисаног frontend-а јесте визуелизација процесног модела у облику интерактивног графа. Визуелизација се заснива на граф репрезентацији у којој стања представљају чворове вишег нивоа, транзиције су усмјерене гране између њих, а задаци се приказују као чворови нижег нивоа унутар стања.

На сликама и приказани су различити нивои детаља процесног графа. Први приказује цјелокупан граф са свим стањима, транзицијама и задацима, док се други фокусира на структуру чворова и грана, истичући логичке везе између елемената.



Слика 5: Визуелизација процесног графа — цјелокупан граф



Слика 6: Визуелизација процесног графа — чворови и гране

Оваква репрезентација омогућава интуитивно праћење тока процеса, јасно истицање гранања и конвергенције токова, идентификацију паралелних активности и визуелни приказ fork-join семантике. Граф се генерише статички у фази генерације кода и користи библиотеке за аутоматско позиционирање чворова.

4.4 Интеграција генерисаних система

Спојивост генерисаног backend-а и frontend-а представља једну од кључних предности BPML приступа. Оба система генеришу се из истог модела, што обезбјеђује структурну конзистентност на свим нивоима.

4.4.1 Типска конзистентност

Типски систем BPML језика представља јединствен извор истине за типове података у оба слоја. Исти доменски тип из BPML модела мапира се на одговарајући Java тип у backend-у и TypeScript тип у frontend-у. Ова кореспонденција елиминише честу категорију грешака у full-stack развоју, а то је неусклађеност типова између клијентске и серверске стране.

На примјер, атрибут `amount: float` из BPML модела мапира се на `Float` у Java ентитету, на `number` у TypeScript интерфејсу, на нумеричко поље у React форми и на одговарајућу колону у бази података. Ова конзистентност постиже се аутоматски, без потребе за ручном координацијом.

4.4.2 API контракт

REST интерфејс генерисаног backend-а и сервисни слој генерисаног frontend-а дијеле заједничку структуру. Endpoint путање, формат захтјева и формат одговора усклађени су на нивоу генерације кода. За сваки CRUD ендпоинт у backend контролеру постоји одговарајућа функција у frontend сервису, чиме се обезбјеђује компатибилност комуникације без додатне конфигурације.

4.4.3 Ток података

Типичан ток интеракције у генерисаном систему одвија се на сљедећи начин: корисник покреће акцију у React компоненти (нпр. креирање ентитета или извршавање транзиције), frontend сервис формира HTTP захтјев и шаље га на одговарајући REST endpoint, backend контролер прослјеђује захтјев сервисном слоју, сервис извршава пословну логику и ажурира базу података, а одговор се враћа кроз исте слојеве до корисничког интерфејса који се ажурира.

У случају процесних операција, ток је сложенији: покретање процесне инстанце укључује креирање инстанце, повезивање ентитета, креирање почетних задатака и евентуално аутоматско завршавање задатака без зависности. Извршавање транзиције укључује валидацију стања, провјеру улоге, биљежење историје, промјену стања, завршетак задатака претходног стања и креирање нових задатака за одредишно стање. Све ове операције извршавају се у оквиру једне трансакције, чиме се обезбјеђује атомичност промјена.

4.5 Предности аутоматске генерације кода

Аутоматска генерација кода из BPML модела доноси низ конкретних предности у односу на ручни развој full-stack апликација.

Конзистентност архитектуре представља прву кључну предност: сви генерисани ентитети, сервиси и компоненте прате исту архитектурну структуру, чиме се елиминише проблем различитих приступа и стилова који се јављају при ручном развоју у тимском окружењу.

Значајна уштеда времена произилази из елиминације ручног писања инфраструктурног кода. Један BPML ентитет са неколико атрибута резултира генерисањем десетак фајлова у backend-у (ентитет, репозиторијум, сервис, контролер, DTO, мапер, евентуално enum класе) и неколико фајлова у frontend-у (TypeScript интерфејс, три React компоненте, сервис), што представља стотине линија кода.

Типска сигурност обезбијеђена је у оба слоја система, статичким типовима у Java backend-у и TypeScript frontend-у, а конзистентност типова гарантована је на нивоу генерације.

Модел као централни извор истине омогућава да се измјене у спецификацији процеса досљедно пропагирају кроз цијели систем поновном генерацијом. Ово елиминише ризик од одступања између спецификације и имплементације, што представља честу категорију проблема у софтверским пројектима.

Брза изградња прототипа такође представља значајну предност: на основу релативно кратке BPML спецификације могуће је у кратком временском року добити функционалан систем са CRUD операцијама, процесним runtime-ом и корисничким интерфејсом.

Глава 5

Валидација и анализа система

У овом поглављу анализира се исправност, функционалност и ограничења предложеног BPML приступа. За разлику од претходних поглавља у којима је систем описан на концептуалном и архитектурном нивоу, овдје се приступ валидира кроз конкретну студију случаја — моделовање и генерисање система за одобравање фактура. Анализа обухвата демонстрацију генерисаног система, верификацију семантичких провјера на примјерима невалидних модела, поређење са постојећим приступима и критичку оцјену предности и ограничења.

5.1 Студија случаја: систем одобравања фактура

За потребе валидације одабран је процес одобравања фактура, који представља типичан пословни сценарио са више учесника, вишеструким нивоима одобравања, паралелним активностима и гранањем одлука. Овај процес одабран је јер покрива све кључне механизме BPML језика: ентитете са различитим типовима атрибута, хијерархију улога, вишеструка стања и транзиције, зависности задатака и аутоматске задатке.

5.1.1 Опис пословног сценарија

Процес одобравања фактура одвија се кроз сљедеће фазе: запослени креира фактуру и подноси је на одобравање, менаџер прегледа и доноси одлуку (одобрава или одбија), финансијски тим врши додатну провјеру, главни финансијски директор (CFO) даје завршно одобрење, након чега слиједи обрада плаћања и завршетак процеса. У сваком кораку одлучивања постоји могућност одбијања фактуре, што резултира преласком у стање одбијања.

Овај сценарио укључује четири улоге организоване у хијерархијску структуру, девет стања у животном циклусу процеса, тринаест задатака (укључујући аутоматске задатке и задатке са зависностима) и десет транзиција (укључујући алтернативне токове за одбијање).

5.1.2 BPML спецификација

Сљедећи листинг приказује комплетну BPML спецификацију процеса одобравања фактура:

```
project {
    name: InvoiceApprovalSystem
    description: "Automated invoice approval workflow
                  with multi-level authorization"
```

```
        version: "1.0.0"
        author: "BPML Team"
    }

    process InvoiceApproval {
        entity Invoice {
            invoiceNumber: string
            vendor: string
            amount: float
            description: string
            invoiceDate: string
            dueDate: string
            category: enum { SUPPLIES, SERVICES,
                             EQUIPMENT, TRAVEL }
            attachmentUrl: string
        }

        entity ApprovalRecord {
            approver: string
            decision: enum { APPROVED, REJECTED, PENDING }
            comments: string
            timestamp: string
        }

        entity PaymentInfo {
            paymentMethod: enum { BANK_TRANSFER,
                                  CHECK, CREDIT_CARD }
            paymentDate: string
            transactionId: string
        }

        role Employee
        role Manager supervises Employee
        role FinanceTeam
        role CFO supervises Manager, FinanceTeam

        state DraftState
        state SubmittedForApprovalState
        state ManagerReviewState
        state FinanceReviewState
        state CFOApprovalState
        state ApprovedState
        state RejectedState
        state PaymentProcessingState
        state CompletedState
```

```
task InitTask in DraftState by Employee
task CreateInvoice in DraftState
    by Employee affects Invoice
task SubmitInvoice in DraftState auto
    affects Invoice
    depends_on InitTask, CreateInvoice
task ReviewByManager in SubmittedForApprovalState
    by Manager affects Invoice, ApprovalRecord
task ReviewByFinance in FinanceReviewState
    by FinanceTeam
    affects Invoice, ApprovalRecord
task ApprovalByCFO in CFOApprovalState
    by CFO affects Invoice, ApprovalRecord
task ProcessPayment in PaymentProcessingState
    by CFO affects PaymentInfo
task RecordApproval in ManagerReviewState
    by Manager affects ApprovalRecord
    depends_on ReviewByManager
task RecordFinanceApproval in FinanceReviewState
    by FinanceTeam affects ApprovalRecord
    depends_on ReviewByFinance
task RecordCFOApproval in CFOApprovalState
    by CFO affects ApprovalRecord
    depends_on ApprovalByCFO
task RejectInvoice in RejectedState
    by Manager affects Invoice
task CloseInvoice in CompletedState
    by CFO affects Invoice, PaymentInfo
    depends_on ProcessPayment
task NotifyStakeholders in CompletedState auto
    depends_on CloseInvoice

transition InitiateDraftTransition
    from DraftState
    to SubmittedForApprovalState by Employee
transition SendToManagerTransition
    from SubmittedForApprovalState
    to ManagerReviewState by Employee
transition ManagerApprovesTransition
    from ManagerReviewState
    to FinanceReviewState by Manager
transition ManagerRejectsTransition
    from ManagerReviewState
    to RejectedState by Manager
transition FinanceApprovesTransition
    from FinanceReviewState
```

```
            to CFOApprovalState by FinanceTeam
        transition FinanceRejectsTransition
            from FinanceReviewState
            to RejectedState by FinanceTeam
        transition CFOApprovesTransition
            from CFOApprovalState
            to ApprovedState by CFO
        transition CFORejectsTransition
            from CFOApprovalState
            to RejectedState by CFO
        transition SendToPaymentTransition
            from ApprovedState
            to PaymentProcessingState by CFO
        transition CompleteProcessTransition
            from PaymentProcessingState
            to CompletedState by CFO
    }
```

Листинг 12: BPML спецификација процеса одобравања фактура

Овај модел илуструје неколико кључних карактеристика BPML језика. Прво, три ентитета (`Invoice`, `ApprovalRecord`, `PaymentInfo`) демонстрирају типски систем: основне типове (`string`, `float`) и набројиве типове (`enum`). Друго, хијерархија улога показује однос надређености — `CFO` надгледа и `Manager` и `FinanceTeam`, док `Manager` надгледа `Employee`. Треће, задатак `SubmitInvoice` илуструје механизам зависности (`depends_on InitTask, CreateInvoice`) и аутоматско извршавање (`auto`). Четврто, транзиције демонстрирају гранање — из стања `ManagerReviewState` процес може прећи у `FinanceReviewState` (одобрење) или `RejectedState` (одбијање).

5.2 Демонстрација генерисаног система

На основу приказане BPML спецификације, покретањем генератора производи се комплетан full-stack систем. У наставку су описани кључни елементи генерисаног система и функционални сценарији којима је потврђена исправност.

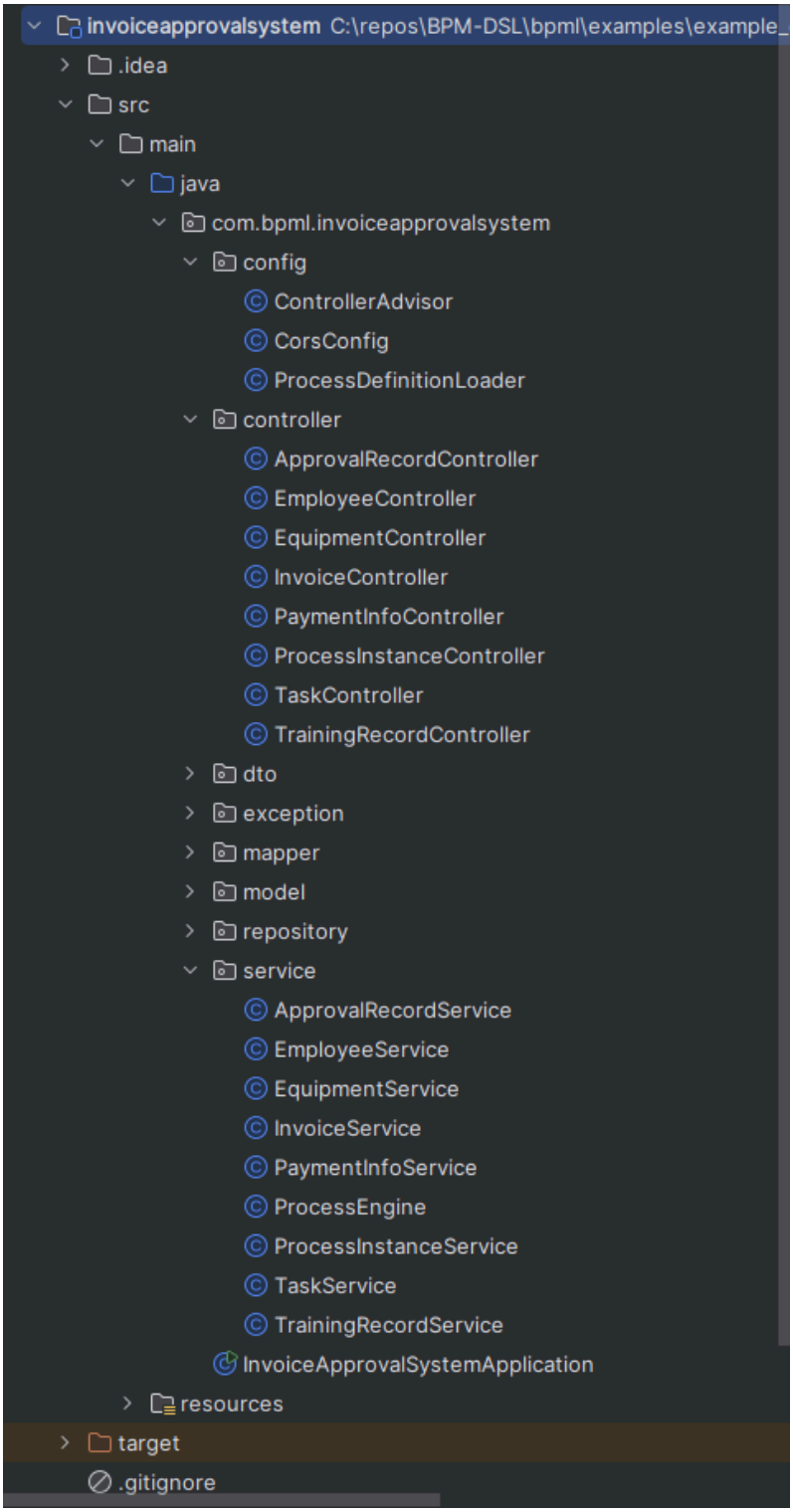
5.2.1 Генерисана backend структура

Spring Boot генератор за модел `InvoiceApprovalSystem` производи сљедеће компоненте:

- три JPA ентитета (`Invoice`, `ApprovalRecord`, `PaymentInfo`) са одговарајућим анотацијама и типским мапирањем,
- три Java `enum` класе (`InvoiceCategory`, `ApprovalRecordDecision`, `PaymentInfoPaymentMethod`) генерисане из набројивих типова,
- три Spring Data репозиторијума,
- три сервисне класе са CRUD операцијама,

- три REST контролера,
- процесну машину (`ProcessEngine`) са статички дефинисаном конфигурацијом стања и транзиција,
- сервисе за управљање процесним инстанцама и задацима,
- DTO класе, мапере и механизам обраде изузетака.

На слици 7 приказана је структура генерисаног Spring Boot пројекта са два процеса, која одражава организацију компоненти у складу са слојевима архитектуре (ентитети, репозиторијуми, сервиси, контролери, процесна логика).

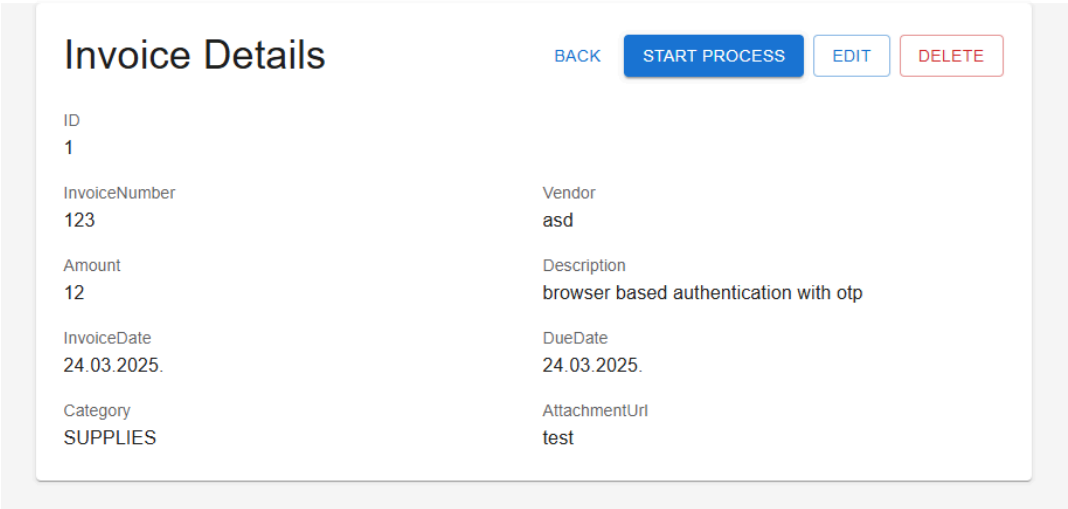


Слика 7: Структура генерисаног Spring Boot пројекта

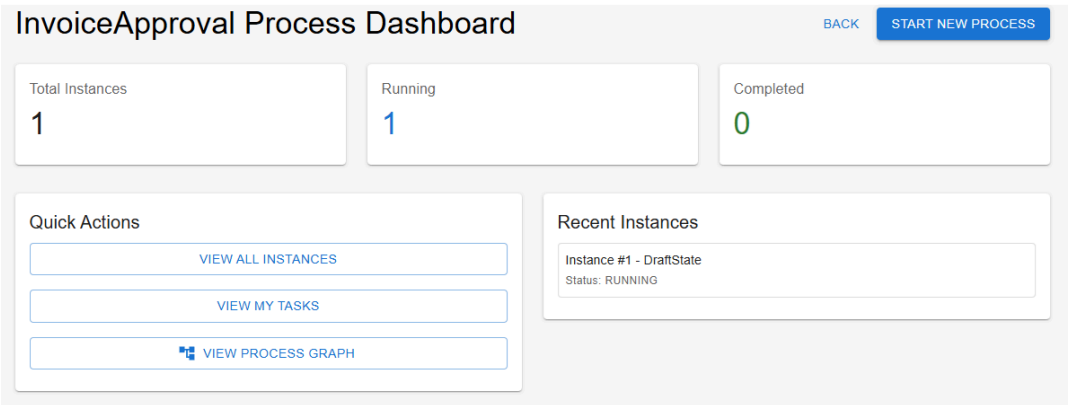
5.2.2 Генерисана frontend структура

React генератор производи одговарајуће frontend компоненте: три сета CRUD компоненти (листа, дијалог, детаљна страница) за сваки ентитет, TypeScript интерфејсе за све ентитете, сервисни слој за комуникацију са backend API-јем, компоненте за управљање процесом (dashboard, листа инстанци, детаљна страница инстанце, листа задатака) и визуелизацију процесног графа.

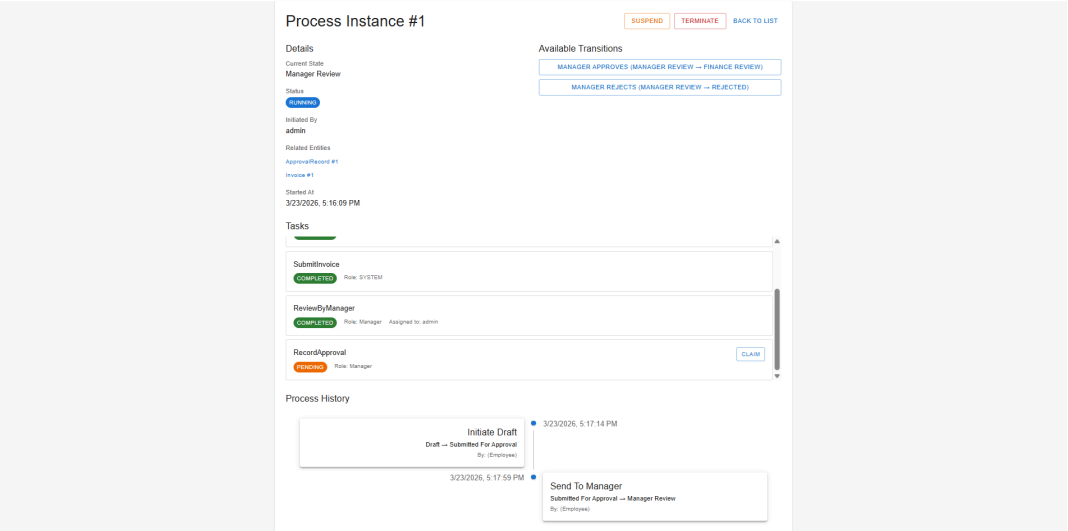
На сљедећим сликама приказани су примјери генерисаних интерфејса: CRUD страница за ентитет `Invoice` (слика 8), dashboard странице процеса са листом активних инстанци (слика 9) и детаљна страница процесне инстанце која приказује тренутно стање, историју транзиција и активне задатке (слика 10).



Слика 8: Генерисана CRUD страница (примјер ентитета Invoice)



Слика 9: Dashboard странице процеса



Слика 10: Страница процесне инстанце

5.2.3 Функционални сценарији

Исправност генерисаног система провјерена је извршавањем комплетног тока процеса кроз сљедеће сценарије:

Сценарио 1: Креирање ентитета и покретање процеса. Корисник са улогом `Employee` креира нову инстанцу ентитета `Invoice` путем CRUD интерфејса, а затим покреће нову процесну инстанцу `InvoiceApproval`. Систем поставља инстанцу у почетно стање `DraftState` и креира три задатка за то стање: `InitTask` и `CreateInvoice` (ручни, додијељени улози `Employee`) и `SubmitInvoice` (аутоматски, са зависностима). Провјерено је да аутоматски задатак остаје у статусу `PENDING` све док зависности нису испуњене.

Сценарио 2: Извршавање задатака са зависностима. Корисник завршава задатке `InitTask` и `CreateInvoice`. Након завршетка оба задатка, систем аутоматски детектује да су зависности задатка `SubmitInvoice` испуњене и аутоматски га завршава. Овим је потврђена коректност механизма каскадне провјере зависности.

Сценарио 3: Извршавање транзиције уз провјеру улоге. Корисник са улогом `Employee` извршава транзицију `InitiateDraftTransition` из стања `DraftState` у `SubmittedForApprovalState`. Покушај извршавања транзиције `ManagerApprovesTransition` од стране корисника са улогом `Employee` резултира грешком. Систем исправно одбија захтјев јер та транзиција захтијева улогу `Manager`. Овим је потврђена исправност контроле приступа засноване на улогама.

Сценарио 4: Алтернативни токови. Из стања `ManagerReviewState` корисник са улогом `Manager` извршава транзицију `ManagerRejectsTransition`, чиме процес прелази

у `RejectedState`. Провјерено је да систем исправно обрађује и алтернативне токове процеса (одбијање), а не само примарни ток (одобрење).

Сценарио 5: Завршетак процеса. Процес пролази комплетан ток одобравања до стања `CompletedState`. Корисник са улогом `CFO` завршава задатак `CloseInvoice`, након чега се аутоматски завршава задатак `NotifyStakeholders` (јер има зависност на `CloseInvoice` и означен је као `auto`). Систем детектује да се процес налази у завршном стању и да су сви задаци завршени, те аутоматски поставља статус инстанце на `COMPLETED`.

Сви описани сценарији успјешно су извршени, чиме је потврђена функционална исправност генерисаног система.

5.3 Верификација семантичких провјера

Семантички слој BPML система описан је у трећем поглављу. Овдје се исправност тих провјера демонстрира на конкретним примјерима невалидних модела и одговарајућих порука о грешци.

5.3.1 Дупликат ентитета

Уколико модел садржи два ентитета са истим називом унутар истог процеса, систем пријављује грешку:

```
process InvoiceApproval {
    entity Invoice { invoiceNumber: string }
    entity Invoice { vendor: string }       // грешка
    ...
}
```

Листинг 13: Невалидан модел — дупликат ентитета

Систем пријављује: `Duplicate entity name 'Invoice' in process 'InvoiceApproval'`. Аналогна провјера спроводи се за дупликате улога, стања, задатака и транзиција.

5.3.2 Самонадгледање улоге

Уколико улога покуша да надгледа саму себе, систем детектује грешку:

```
process InvoiceApproval {
    ...
    role Manager supervises Manager    // грешка
    ...
}
```

Листинг 14: Невалидан модел — самонадгледање улоге

Систем пријављује: `Role 'Manager' cannot supervise itself in process 'InvoiceApproval'`.

5.3.3 Транзиција са идентичним стањима

Уколико транзиција повезује исто полазно и одредишно стање, систем одбија модел:

```
process InvoiceApproval {
    ...
    state DraftState
    ...
    transition InvalidTransition
        from DraftState
        to DraftState                  // грешка
        by Employee
}
```

Листинг 15: Невалидан модел — транзиција са идентичним стањима

Систем пријављује: `Transition 'InvalidTransition' has same from and to state 'DraftState' in process 'InvoiceApproval'.`

5.3.4 Задатак са нарушеним ексклузивитетом

BPML језик намеће правило да задатак мора бити или ручни (додијељен улози) или аутоматски, али не обоје истовремено:

```
process InvoiceApproval {
    ...
    task InvalidTask in DraftState
        by Employee auto              // грешка
    ...
}
```

Листинг 16: Невалидан модел — задатак који је истовремено ручни и аутоматски

Систем пријављује: `Task 'InvalidTask' must be either automated (use 'auto') or assigned to a role (use 'by RoleName') in process 'InvoiceApproval'.`

5.3.5 Задатак који зависи од самог себе

Систем детектује директне циркуларне зависности у задацима:

```
process InvoiceApproval {
    ...
    task ReviewByManager in ManagerReviewState
        by Manager
        depends_on ReviewByManager     // грешка
    ...
}
```

Листинг 17: Невалидан модел — циркуларна зависност задатка

Систем пријављује: `Task 'ReviewByManager' cannot depend on itself in process 'InvoiceApproval'.`

5.3.6 Невалидна референца у транзицији

Уколико транзиција референцира улогу која не постоји у процесу, систем пријављује грешку:

```
process InvoiceApproval {
    ...
    role Employee
    ...
    transition InvalidTransition
        from DraftState
        to SubmittedForApprovalState
        by Admin                        // грешка
    ...
}
```

Листинг 18: Невалидан модел — непостојећа улога у транзицији

Систем пријављује: `Transition 'InvalidTransition' references unknown role 'Admin' in process 'InvoiceApproval'`.

Сви наведени примјери потврђују да семантички слој BPML система детектује грешке у фази обраде модела, прије покретања генерације кода. На тај начин се обезбјеђује да генерисани систем одговара искључиво валидним и конзистентним моделима.

5.4 Анализа предности приступа

На основу спроведене студије случаја и функционалне верификације, могу се уочити сљедеће конкретне предности BPML приступа.

Централизација процесне логике у DSL-у омогућава да се цијели процес одобравања фактура, укључујући девет стања, десет транзиција, тринаест задатака и четири улоге, дефинише у једном текстуалном фајлу. Измјена процеса (нпр. додавање новог нивоа одобравања) захтијева промјену искључиво у BPML спецификацији, а не у десетинама генерисаних фајлова.

Елиминација инфраструктурног кода представља значајну уштеду. Из приказане спецификације генерише се комплетан backend систем (JPA ентитети, репозиторијуми, сервиси, контролери, DTO класе, процесна машина, обрада грешака, конфигурација) и комплетан frontend систем (TypeScript интерфејси, CRUD компоненте, процесне компоненте, сервисни слој, рутирање). Ручна имплементација еквивалентног система захтијевала би вишеструко више времена и кода.

Конзистентност између модела и имплементације гарантована је механизмом генерације. Уколико се у BPML моделу дефинише транзиција `ManagerApprovesTransition` из `ManagerReviewState` у `FinanceReviewState` од стране улоге `Manager`, генерисана

процесна машина гарантује да се та транзиција може извршити искључиво у тим условима. Не постоји могућност одступања имплементације од спецификације.

Рано откривање грешака путем семантичке валидације, демонстрирано у претходној секцији, спречава генерисање некоректног кода. Грешке као што су дупликати, невалидне референце и нарушена правила детектују се пријe покретања генерације.

Перформансе генерисаног система бенефицирају од статичке генерације: процесна логика је интегрисана у апликациони код и не захтијева runtime интерпретацију DSL-а. Скалабилност зависи од конфигурације Spring Boot апликације и базе података, а не од засебног engine механизма.

5.5 Поређење са постојећим приступима

Ради контекстуализације BPML приступа, у наставку је дато структурирано поређење са два алтернативна приступа: интерпретативним BPM платформама и ручном имплементацијом.

Табела 2: Поређење BPML приступа са алтернативним приступима

Критеријум	**BPML (генеративни DSL)**	**Camunda / Activiti**	**Ручна имплементација**
Тип приступа	Генеративни	Интерпретативни	Ад-хок
Спецификација процеса	Формална (DSL)	BPMN дијаграм	Имплицитна у коду
Генерација кода	Комплетна (backend + frontend)	Дјелимична (процесни слој)	Не постоји
Runtime engine	Није потребан	Захтијева засебну инфраструктуру	Није потребан
Динамичка измјена процеса	Захтијева регенерацију	Подржана	Захтијева измјену кода
Крива учења	BPML синтакса	BPMN + конфигурација engine-а	Зависи од технологија
Семантичка валидација	Статичка, прије генерације	Дјелимична	Не постоји (мануелна)
Визуелизација процеса	Генерисана (граф)	Интегрисана (BPMN едитор)	Не постоји
Типска сигурност	Кроз цијели стек	Дјелимична	Зависи од имплементације
Скалабилност	Spring Boot стандардна	Зависи од engine-а (Zeebe за дистрибуирано)	Зависи од имплементације

Из поређења приказаног у табели 2 видљиво је да сваки приступ има специфичне предности. Интерпретативне платформе попут Camunda доминирају у сценаријима који захтијевају динамичку измјену процеса без прекида рада система, зрео екосистем за мониторинг и аналитику, те интеграцију са постојећим BPMN моделима. Ручна имплементација пружа максималну флексибилност, али уз значајно веће улагање времена и без формалне гаранције конзистентности.

BPML приступ показује предности у сценаријима гдје је потребна брза изградња комплетних full-stack система на основу формалне спецификације, гдје се процеси мијењају релативно ријетко (нпр. фаза развоја прототипа или системи са стабилним процесима) и гдје је важна типска конзистентност кроз цијели технолошки стек.

5.6 Ограничења и потенцијална побољшања

Упркос наведеним предностима, анализом система идентификована су одређена ограничења која представљају основу за будући развој.

Одсуство условних израза у транзицијама представља значајно ограничење. Тренутно, BPML не подржава формалне услове за транзиције (нпр. `if amount > 10000 then CFOApproval`). Одлуке о гранању процеса зависе искључиво од корисничке интеракције, односно корисник одговарајуће улоге одлучује коју транзицију ће извршити. Увођење условних израза омогућило би аутоматско одлучивање на основу вриједности атрибута ентитета, чиме би се повећала аутоматизација процеса.

Ограничена подршка за сложене типове података представља друго ограничење. Тренутно су подржани само основни типови (`int`, `float`, `string`, `boolean`) и набројиви типови. Не постоји подршка за датумске типове, релације између ентитета (нпр. `one-to-many`, `many-to-many`) или угнијеждене структуре. Увођење датумских типова и односа између ентитета значајно би проширило домен примјене.

Одсуство динамичке конфигурације процеса произилази из генеративног приступа. Свака измјена процеса захтијева поновну генерацију и deployment апликације. Ово ограничење је свјесна дизајнерска одлука, али у одређеним сценаријима (нпр. честе измјене процеса у продукционом окружењу) може представљати недостатак у поређењу са интерпретативним системима.

Непостојање графичког едитора модела захтијева од корисника познавање текстуалне синтаксе BPML језика. Овај недостатак дјелимично је ублажен развојем VSCode екстензије која обезбјеђује синтаксно бојење, исјечке кода и аутоматско довршавање, чиме се значајно олакшава рад са текстуалном синтаксом. Ипак, не постоји обрнути смјер, тј. не постоји могућност креирања BPML спецификације путем графичког интерфејса. Развој графичког едитора који генерише BPML код представља значајан правац будућег развоја.

Одсуство напредних аналитичких механизама такође представља ограничење. Генерисани систем биљежи историју транзиција, али не обезбјеђује алате за анализу перформанси процеса, идентификацију уских грла или оптимизацију тока. Интеграција аналитичких механизама, потенцијално инспирисаних концептом *process mining*-а [9], представља додатни правац будућег рада.

5.7 Општа оцјена система

На основу спроведене студије случаја, функционалне верификације и критичке анализе, може се закључити да предложени BPML приступ успјешно остварује постављене циљеве рада. Систем омогућава декларативну спецификацију пословних процеса,

статичку семантичку валидацију модела, аутоматску генерацију комплетне full-stack апликације, укључујући процесну машину и визуелизацију процесног графа.

Студија случаја показала је да се реалистичан пословни процес (систем одобравања фактура са четири улоге, девет стања и тринаест задатака) може у потпуности спецификовати у BPML језику и аутоматски трансформисати у функционалан софтверски систем. Семантичке провјере детектују грешке у раној фази, а генерисани систем досљедно примјењује правила дефинисана моделом.

Идентификована ограничења (одсуство условних израза, сложених типова, динамичке конфигурације и графичког едитора) представљају конкретне правце за будући развој језика и не умањују вриједност демонстрираног приступа у контексту брзе изградње прототипова и система са стабилним пословним процесима.

Глава 6

Закључак

У овом раду представљен је језик специфичан за домен BPML (Business Process Modeling Language), намијењен моделовању пословних процеса и аутоматској генерацији full-stack софтверских система. Основна идеја рада заснива се на централизацији процесне логике у декларативном моделу, који се затим трансформише у извршиву backend и frontend апликацију.

У оквиру рада реализовани су сљедећи кључни елементи:

- дефинисање формалне граматике BPML језика употребом TextX алата, укључујући 23 кључне ријечи распоређене у четири семантичке категорије,
- имплементација статичке семантичке валидације модела која обухвата провјеру јединствености назива, исправности референци, конзистентности хијерархије улога и зависности задатака,
- развој механизма генерације кода заснованог на Jinja2 шаблонском механизму, са прилагођеним филтерима за типско мапирање и форматирање идентификатора,
- генерисање backend система заснованог на Spring Boot платформи, укључујући JPA ентитете, сервисни слој, REST интерфејс и процесну машину,
- генерисање frontend система заснованог на React и TypeScript технологијама, укључујући CRUD компоненте, управљање процесима и визуелизацију процесног графа,
- имплементација процесне машине за управљање животним циклусом процесних инстанци, са подршком за контролу стања, валидацију транзиција, управљање задацима и аутоматско извршавање,
- развој VSCode екстензије за подршку BPML језику, укључујући синтаксно бојење, исјечке кода за брзо креирање конструкција и конфигурацију едитора.

Предложени приступ омогућава да се једном дефинисана спецификација користи као централни извор истине за цјелокупан систем. На тај начин се обезбјеђује конзистентност између модела и имплементације, смањује количина ручно писаног инфраструктурног кода и убрзава развој нових система. Типска конзистентност гарантована је кроз цијели технолошки стек. Исти доменски тип из BPML модела досљедно се мапира на одговарајуће Java и TypeScript типове.

Валидација система спроведена је кроз студију случаја — моделовање процеса одобравања фактура са четири улоге, девет стања и тринаест задатака. Функционално тестирање потврдило је да генерисани систем коректно управља животним циклусом процеса: процесна машина не дозвољава нелегалне транзиције, контрола приступа

заснована на улогама функционише исправно, механизам зависности задатака обезбјеђује коректну fork-join семантику, а аутоматски задаци се завршавају каскадном провјером зависности. Верификација семантичких провјера на примјерима невалидних модела потврдила је да систем детектује грешке у фази обраде модела, прије покретања генерације кода.

Поређење са постојећим приступима показало је да BPML има специфичне предности у сценаријима брзе изградње прототипа и система са стабилним процесима, док интерпретативне платформе попут Camunda доминирају у сценаријима који захтијевају динамичку измјену процеса без прекида рада система.

Ипак, уочена су и одређена ограничења. Тренутна верзија језика не подржава условне изразе у транзицијама, сложене типове података и односе између ентитета, динамичку конфигурацију процеса, графички едитор за моделовање, нити напредне аналитичке механизме. Ова ограничења представљају конкретне правце за будући развој и не умањују вриједност демонстрираног приступа у контексту генеративних DSL система.

Закључно, BPML представља практичну примјену концепата језика специфичних за домен и моделом вођеног развоја у области пословних процеса. Рад показује да је могуће изградити систем који омогућава директну трансформацију формалне спецификације у извршиву full-stack апликацију, чиме се остварује значајна уштеда времена и повећање конзистентности система.

6.1 Будући рад

Будући развој BPML језика може се организовати у три категорије: проширење самог језика, побољшање алатске подршке и интеграција са постојећим екосистемима.

6.1.1 Проширење језика

Приоритетно проширење типског система укључује увођење подршке за датумске и временске типове (`date`, `datetime`). Ово би омогућило природније моделовање доменских података као што су рокови, датуми креирања и периоди важења, те досљедно мапирање на типове циљних платформи (нпр. Java Time API и одговарајуће TypeScript репрезентације).

Увођење условних израза у транзицијама омогућило би аутоматско одлучивање на основу вриједности атрибута ентитета. На примјер, аутоматско усмјеравање фактура изнад одређеног износа на додатни ниво одобравања. Ово проширење захтијева дефинисање синтаксе условних израза у граматици и имплементацију евалуационог механизма у процесној машини.

Подршка за односе између ентитета (`one-to-many`, `many-to-many`) значајно би проширила домен примјене, омогућавајући моделовање реалистичнијих доменских модела.

Ово проширење захтијева измјене у граматици, типском систему, генерацији JPA ентитета и React компоненти.

6.1.2 Алатска подршка

Развој графичког едитора који генерише BPML спецификацију представља значајан правац будућег рада. Графички едитор омогућио би корисницима визуелно моделовање процеса са аутоматским генерисањем одговарајућег BPML кода, чиме би се смањила потреба за познавањем текстуалне синтаксе.

Увођење аналитичких механизама за праћење перформанси процеса, потенцијално инспирисаних концептом *process mining*-а, омогућило би идентификацију уских грла, анализу времена извршавања и оптимизацију токова.

6.1.3 Интеграције

Интеграција са постојећим BPM алатима и стандардима, укључујући могућност увоза BPMN дијаграма и извоза BPML модела у BPMN формат, олакшала би усвајање BPML језика у организацијама које већ користе BPMN нотацију.

Подршка за дистрибуирано извршавање процеса и додатне циљне платформе (нпр. .NET backend или Vue.js frontend) проширила би домен примјене система.

Списак слика

Слика 1 Архитектурни дијаграм BPML система 12
Слика 2 Илустрација fork/join структуре задатака 20
Слика 3 Синтаксно бојење BPML фајла у VSCode едитору 24
Слика 4 UML дијаграм метамодела BPML језика 26
Слика 5 Визуелизација процесног графа — цјелокупан граф 40
Слика 6 Визуелизација процесног графа — чворови и гране 40
Слика 7 Структура генерисаног Spring Boot пројекта 48
Слика 8 Генерисана CRUD страница (примјер ентитета Invoice) 49
Слика 9 Dashboard странице процеса 49
Слика 10 Страница процесне инстанце 50

Списак листинга

Листинг 1 Структура BPML имплементације 13
Листинг 2 Комплетна TextX граматика BPML језика 13
Листинг 3 Формални EBNF приказ синтаксе BPML језика 15
Листинг 4 Структура семантичке валидације 21
Листинг 5 Разматрана алтернатива — експлицитна fork/branch синтакса 22
Листинг 6 VSCode исјечак за креирање транзиције 25
Листинг 7 Формирање контекста за генерацију backend система 30
Листинг 8 Мапирање BPML типова на Java и TypeScript типове 31
Листинг 9 Генерисање backend фајлова за сваки ентитет 32
Листинг 10 Генерисани Java код за извршавање транзиције 35
Листинг 11 Исјечак Jinja шаблона за генерисање React листе ентитета 38
Листинг 12 BPML спецификација процеса одобравања фактура 43
Листинг 13 Невалидан модел — дупликат ентитета 51
Листинг 14 Невалидан модел — самонадгледање улоге 51
Листинг 15 Невалидан модел — транзиција са идентичним стањима 52
Листинг 16 Невалидан модел — задатак који је истовремено ручни и аутоматски . 52
Листинг 17 Невалидан модел — циркуларна зависност задатка 52
Листинг 18 Невалидан модел — непостојећа улога у транзицији 53

Списак табела

Табела 1 Систематизација кључних ријечи BPML језика 17
Табела 2 Поређење BPML приступа са алтернативним приступима 55

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
CSS	Cascading Style Sheets (језик за описивање стилова)
DOM	Document Object Model (објектни модел документа)
DTO	Data Transfer Object (објекат за пренос података)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
RLS	Row-Level Security (сигурност на нивоу реда)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RPC	Remote Procedure Call (позив удаљене процедуре)
SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
UML	Unified Modeling Language (језик за моделовање дијаграма)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WAL	Write-Ahead Logging (записивање операција унапред)

Списак коришћених појмова

Појам	Објашњење
Асихнрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

[1] И. Дејановић, *Језици специфични за домен*. Факултет техничких наука, Нови Сад, 2021.

[2] I. Dejanović, R. Vaderna, G. Milosavljević, и Ž. Vuković, „TextX: A Python tool for Domain-Specific Languages implementation“, *Knowledge-Based Systems*, том 115, 2017, doi: 10.1016/j.knosys.2016.10.023.

[3] M. Fowler, *Domain-Specific Languages*. Addison-Wesley, 2010.

[4] M. Mernik, J. Heering, и A. M. Sloane, „When and How to Develop Domain-Specific Languages“, *ACM Computing Surveys*, том 37, изд. 4, стр. 316–344, 2005, doi: 10.1145/1118890.1118892.

[5] D. C. Schmidt, „Model-Driven Engineering“, *IEEE Computer*, том 39, изд. 2, стр. 25–31, 2006, doi: 10.1109/MC.2006.58.

[6] M. Dumas, M. La Rosa, J. Mendling, и H. Reijers, *Fundamentals of Business Process Management*, 2nd изд. Springer, 2018. doi: 10.1007/978-3-662-56509-4.

[7] M. Weske, *Business Process Management: Concepts, Languages, Architectures*, 2nd изд. Springer, 2012. doi: 10.1007/978-3-642-28616-2.

[8] W. M. P. van der Aalst, „The Application of Petri Nets to Workflow Management“, *Journal of Circuits, Systems, and Computers*, том 8, изд. 1, стр. 21–66, 1998, doi: 10.1142/S0218126698000043.

[9] W. M. P. van der Aalst, *Process Mining: Data Science in Action*, 2nd изд. Springer, 2016. doi: 10.1007/978-3-662-49851-4.

[10] J. C. Cleaveland, „Program Generators with XML and Java“, *IEEE Software*, том 18, изд. 5, стр. 25–31, 2001, doi: 10.1109/52.951493.